



Universidade Federal do Rio Grande do Norte

Centro de Ciências Exatas e da Terra

Curso de Graduação em Estatística

Um guia para agrupamento com pacote *cluster*
do *R* utilizando dados do *spotify*

Samuel Ricardo Nobre Duarte

Natal, Setembro

2021

Samuel Ricardo Nobre Duarte

Um guia para agrupamento com pacote *cluster* do *R* utilizando dados do *spotify*

Monografia de Graduação apresentada ao
Curso de Graduação em Estatística do
Centro de Ciências Exatas e da Terra da
Universidade Federal do Rio Grande do
Norte como requisito parcial para a
obtenção do grau de Bacharel em Estatística.

Universidade Federal do Rio Grande do Norte

Centro de Ciências Exatas e da Terra

Curso de Graduação em Estatística

Orientadora: Profa. Dra. Carla Almeida Vivacqua

Natal, Setembro

2021

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Setorial Prof. Ronaldo Xavier de Arruda - CCET

Duarte, Samuel Ricardo Nobre.

Um guia para agrupamento com pacote cluster do R utilizando dados do spotify / Samuel Ricardo Nobre Duarte. - 2021.
81f.: il.

Monografia (Bacharelado em Estatística) - Universidade Federal do Rio Grande do Norte, Centro de Ciências Exatas e da Terra, Departamento de Estatística. Natal, 2021.

Orientadora: Profa. Dra. Carla Almeida Vivacqua.

1. Estatística - Monografia. 2. Distância - Monografia. 3. Música - Monografia. 4. Playlist - Monografia. 5. Streaming - Monografia. 6. Similaridade - Monografia. 7. Tutorial - Monografia. I. Vivacqua, Carla Almeida. II. Título.

RN/UF/CCET

CDU 519.2

Samuel Ricardo Nobre Duarte

Um guia para agrupamento com pacote *cluster* do *R* utilizando dados do *spotify*

Monografia de Graduação apresentada ao Curso de Graduação em Estatística do Centro de Ciências Exatas e da Terra da Universidade Federal do Rio Grande do Norte como requisito parcial para a obtenção do grau de Bacharel em Estatística.

Trabalho aprovado, Natal-RN, setembro de 2021

Profa. Dra. Carla Almeida Vivacqua

Orientadora

Prof. Dr. Marcus Alexandre Nunes

Examinador

Prof. Dr. Bruno Monte de Castro

Examinador

Natal, Setembro

2021



Emitido em 23/09/2021

ATA DE DEFESA DE TRABALHO DE CONCLUSÃO DE CURSO Nº 10/2021 - EST/CCET (12.02)

(Nº do Protocolo: NÃO PROTOCOLADO)

(Assinado digitalmente em 23/09/2021 19:31)

BRUNO MONTE DE CASTRO
PROFESSOR DO MAGISTERIO SUPERIOR
EST/CCET (12.02)
Matrícula: 2354162

(Assinado digitalmente em 24/09/2021 05:14)

CARLA ALMEIDA VIVACQUA
PROFESSOR DO MAGISTERIO SUPERIOR
EST/CCET (12.02)
Matrícula: 1218831

(Assinado digitalmente em 23/09/2021 16:53)

MARCUS ALEXANDRE NUNES
PROFESSOR DO MAGISTERIO SUPERIOR
EST/CCET (12.02)
Matrícula: 1066308

Para verificar a autenticidade deste documento entre em <https://sipac.ufrn.br/documentos/> informando seu número:
10, ano: **2021**, tipo: **ATA DE DEFESA DE TRABALHO DE CONCLUSÃO DE CURSO**, data de emissão: **23/09**
/2021 e o código de verificação: **e25ea475f6**

Agradecimentos

Agradeço a Deus por tudo o que ele tem feito em minha vida, dando saúde e discernimento para alcançar os meus objetivos.

Ao meu pai Sergio Ricardo Duarte e à minha mãe Jailma Karla Nobre Duarte que com amor e carinho me educaram e apoiaram em todos os momentos, a toda minha família agradeço por toda ajuda e apoio.

Aos colegas de graduação que persistiram em suas jornadas e, em especial, aos meus amigos Mariana da Silva Cavalcante, Luí Vítor Bezerril Rocha e Matheus Borja Souza, que estiveram nos momentos mais difíceis da minha jornada. Aos meus amigos Maria Kely Alves Gomes da Silva e Emanuel Montenegro Ferreira pela amizade, confiança e momentos de conversa. Aos colegas de turma Fabrício, Rayland, Carla, Ivonaldo, Jaylhane e todos que compartilharam disciplinas comigo.

Aos meus companheiros que partilharam meu período de estágio no SEBRAE/RN e, em especial, a todos da Unidade de Desenvolvimento Setorial que contribuíram para minha formação como profissional, me expondo a situações em que exerci conhecimentos da minha área e também habilidades de comunicação e trabalho em grupo.

Ao departamento de Estatística e a todos os professores que exercem um ensino de excelência conduzindo cada disciplina com rigor. Em especial a Profa. Dra. Carla Almeida Vivacqua que me orientou com dedicação e cuidado para a realização deste trabalho.

A todos que vivenciaram esses dias, meu sincero obrigado.

O Sonho das pessoas não tem fim

Marshall D. Teach

Resumo

Uma forma comum de compreender problemas e questões do dia a dia é agrupar objetos de acordo com suas características. A importância disso está ligada à compreensão de similaridades, ajudando a formular hipóteses. Com isso, o objetivo deste trabalho é desenvolver um guia para auxiliar o processo de criação de grupos. As etapas consideradas são: estabelecimento de um objetivo para realizar o agrupamento; escolha de técnicas de agrupamento, explorando suas construções utilizando conceitos de distância e matriz de dissimilaridades; avaliação de um número adequado de grupos; e interpretação do agrupamento. A aplicação neste trabalho aborda a arte musical por meio de métricas. O Spotify disponibiliza pela sua API dados referentes às músicas do aplicativo. Utiliza-se este serviço de *streaming* de áudio para ilustrar diversas técnicas de clusterização disponíveis no pacote *cluster* da linguagem de programação *R*, explorando caminhos do tópico de agrupamento, construindo perfis, analisando características dos grupos, além de avaliar possíveis diferenças entre os grupos.

Palavras-chave: Distância; Música; *Playlist*; *Streaming*; Similaridade; Tutorial.

Abstract

A common way to understand everyday problems and issues is to group objects according to their characteristics. The importance of this is linked to the understanding of similarities, helping to formulate hypotheses. Thus, the objective of this work is to develop a guide to help the process of creating groups. The steps considered are: establishment of an objective to carry out the grouping; choice of grouping techniques, exploring their constructions using concepts of distance and matrix of dissimilarities; evaluation of an adequate number of groups; and interpretation of the grouping. The application in this work approaches the musical art through metrics. Spotify makes available through its API data referring to the songs of the app. This audio streaming service is used to illustrate several clustering techniques available in the R programming language cluster package, exploring paths of the clustering topic, building profiles, analyzing group characteristics, and evaluating possible differences between groups.

Keywords: Distance; Song; Playlist; Streaming; Similarity; Tutorial.

Lista de Ilustrações

FIGURAS

Figura 1 - Histograma das variáveis numéricas do banco de dados extraído do <i>spotify</i>	20
Figura 2 - Heatmap - com as correlações das variáveis numéricas do banco de dados extraído do <i>spotify</i>	22
Figura 3 - Gráficos de Dispersão 2x2 das variáveis numéricas do banco de dados extraído do <i>spotify</i>	22
Figura 4 - Gráfico da função ggpairs da biblioteca GGally das variáveis numéricas.....	23
Figura 5 - Frequência dos tons nas músicas dos dados extraídos do <i>spotify</i>	23
Figura 6 - Frequência dos tipos de organização de músicas no banco de dados.....	24
Figura 7 - Artistas mais frequentes no banco de dados com mais de 20 músicas presentes.....	24
Figura 8 - Quantidade de conteúdo explícito no banco de dados.....	25
Figura 9 - Fluxograma para realização de agrupamento.....	27
Figura 10 - Método da Silhueta aplicado a função pam para as variáveis Instrumentabilidade e Dançabilidade.....	45
Figura 11 - Gráfico de dispersão 2x2 com a clusterização realizada pela função pam.....	45
Figura 12 - Método da Silhueta aplicado a função clara para as variáveis numéricas do banco de dados.....	50
Figura 13 - Intervalos do teste de tukey para cada par de cluster variando para cada variável numérica.....	51
Figura 14 - Exemplo de pontos em regiões específicas em que algoritmos de clusterização tem dificuldade em agrupar.....	54
Figura 15 - Clusterização dos tons musicais do banco de dados com as músicas do <i>spotify</i> com a função Fanny.....	57
Figura 16 - Gráfico da Silhueta dos <i>clusters</i> gerados pela função Fanny.....	57
Figura 17 - Gráfico tipo Banner da função Agnes para o conjunto de dados.....	62

Figura 18 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados.....	63
Figura 19 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados com método single.....	63
Figura 20 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados com método complete.....	64
Figura 21 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados com método flexible, com parâmetros fixados em 0.9.....	64
Figura 22 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados com método gaverage.....	65
Figura 23 - Gráfico tipo Banner da função Diana para o conjunto de dados.....	68
Figura 24 - Gráfico tipo Dendrograma para a função Diana aplicado às médias das <i>playlists</i> para cada atributo.....	69
Figura 25 - Gráfico tipo banner com aplicação da função Mona aos dados binários.....	74

QUADROS

Quadro 1 - Lista de variáveis selecionadas para compor aplicação.....	18
Quadro 2 - Sumarização da Manova para função Pam.....	46
Quadro 3 - Sumarização da Manova para função Clara.....	50
Quadro 4 - Médias dos <i>clusters</i> gerados pela função clara para cada atributo.....	52
Quadro 5 - Exemplo de tabela de contingência.....	71
Quadro 6 - Critérios utilizados para criação de variáveis binárias.....	72

TABELAS

Tabela 1 - Sumarização das variáveis numéricas.....	20
--	----

Sumário

1. INTRODUÇÃO.....	13
2. OBJETIVOS.....	14
3. REFERENCIAL TEÓRICO.....	15
4. BANCO DE DADOS.....	16
4.1.Web Scraping.....	16
4.2.Seleção de variáveis.....	17
5. ANÁLISE EXPLORATÓRIA.....	19
6. MÉTODOS.....	25
6.1. Métodos Hierárquicas.....	26
6.2. Métodos Não Hierárquicas.....	26
6.3. Distâncias.....	28
6.3.1. Distância Euclidiana.....	28
6.3.2. Distância de Manhattan.....	29
6.3.3. Distância de Gower.....	30
6.3.4. Distância de Jaccard.....	31
6.3.5. Squared Euclidean.....	31
6.3.6.Minkowski.....	31
6.3.7. Canberra.....	32
6.4. Matriz de Dissimilaridade.....	32
6.5. k-means.....	32
6.6. Silhueta.....	33
6.7. Métodos de avaliação.....	34
6.7.1. Manova.....	35
6.7.2. Teste de Tukey.....	36

7. PACOTE CLUSTER.....	36
7.1. DAISY.....	39
7.2. PAM.....	42
7.3. CLARA.....	47
7.4. FANNY.....	53
7.5. AGNES.....	58
7.6. DIANA.....	66
7.7. MONA.....	70
8. CONSIDERAÇÕES FINAIS.....	75
REFERÊNCIAS.....	77
APÊNDICE A - CÓDIGO PARA WEB SCRAPING DOS DADOS DO SPOTIFY.....	78
APÊNDICE B - SUMARIZAÇÃO DA FUNÇÃO FANNY - COM AS SAÍDAS PARA CADA OBJETO GERADO PELO ALGORITMO.....	80
APÊNDICE C - FIGURA EXPLICATIVA COM PLOTAGEM DO BANNER PARA FUNÇÃO DIANA COM O MOMENTO DE DIVISÃO DOS <i>CLUSTERS</i>.....	81

1. INTRODUÇÃO

Na história da humanidade temos diversas situações em que houve a necessidade de classificar e agrupar, seja para sobrevivência ou compreensão de fenômenos. Pode-se citar, por exemplo, uma criança que tem que diferenciar um círculo de um triângulo na infância em uma atividade da escola, ou na biologia em que se classifica que gatos e leões pertencem à mesma família dos felinos, e que, apesar de um gato ser diferente de um leão, visto que letalidade, tamanho, aparência física e outros aspectos diferem, ambos preservam inúmeras características que ajudam a compreender seus comportamentos e diferenças de outros animais, contemplando um nível de similaridade mais próximo do que outros.

Essa necessidade de criar grupos e classificá-los vem ajudando em nosso dia a dia desde os primórdios. Os dados que são abordados para a aplicação de técnicas de agrupamento deste trabalho estão relacionados a um dos tipos de artes mais difundida na nossa rotina, e com eles podemos observar o funcionamento dessas técnicas com a temática, que se demonstra de suma importância para seu cenário. Tendo em vista que arte ao decorrer dos tempos foi construindo sua história e se diferenciando por conta dos seus criadores. A arte sonora, mais popularmente conhecida como música, passou por diversas transformações ao decorrer dos séculos. Na segunda metade do século XX, a prática da comercialização de músicas e de grupos musicais entrou em ascensão ganhando forma, até se tornar a indústria fonográfica que conhecemos hoje. A perspectiva agora dentro do mercado fonográfico é que a música, o grupo ou o artista são itens comerciais, ou seja, o sucesso do produto se baseia em sua popularidade e quanto o artista consegue reverter sua arte em fatores financeiros e midiáticos. Então é de extrema importância para as empresas ligadas à área fonográfica compreender que tipo de produto musical irá ter a maior chance de se tornar um sucesso entre a população, como também é de interesse compreender seu público e suas necessidades.

Com essa perspectiva, métricas e rótulos são essenciais para conseguir agrupar músicas para distribuir para o público certo. Neste trabalho terá enfoque abordar algoritmos de agrupamentos, utilizando como base dados relacionados a características de músicas que são utilizadas pelo aplicativo *spotify*. Será abordado desde a extração desses dados pela *API*

(Interface de programação de aplicações) do *spotify* disponível para desenvolvedores, passando pela explicitação dessas ferramentas dentro do contexto da estatística multivariada até a aplicação dos algoritmos utilizando a linguagem de programação *R* e discutindo seus resultados. Vale ressaltar que a aplicação utilizará uma biblioteca cujo nome é ‘*cluster*’ e que ela é uma das possíveis maneiras de se abordar essa problemática de agrupamento no contexto de análise multivariada.

2. OBJETIVOS

A análise estatística multivariada tem como um dos seus tópicos a classificação e o agrupamento de dados, as ferramentas para a análise apresentam grande utilidade para a compreensão das observações e suas variáveis. De modo geral técnicas de agrupamento ajudam a construir hipóteses de maneira subjetiva, compreender múltiplas dimensionalidades e outros aspectos; O objetivo geral deste trabalho está ligado a compreensão do agrupamento por intermédio de algumas dessas técnicas. No século XX tivemos grandes desenvolvimentos relacionados às técnicas de agrupamentos, isso em grande parte se deve ao avanço tecnológico e com isso a implementação de programas para análise de dados. O livro de Kaufman e Rousseeuw 1990, doravante [KR] implementa diversas técnicas na linguagem de FORTRAN pela biblioteca CLUSFIND. Nesta biblioteca haviam sido implementados sete algoritmos de agrupamento, cada um dos algoritmos com nomes femininos, sendo eles: AGNES, CLARA, DAISY, DIANA, FANNY, MONA e PAM. Temos também o artigo de Struyf, Hubert e Rousseeuw 1997, Antwerp [BE] que incorpora os algoritmos na linguagem de S-PLUS. O objetivo deste trabalho dentro do contexto multivariado é ser um guia do pacote ‘*cluster*’. Ilustrando o uso de técnicas de agrupamento e suas possíveis aplicabilidades, isso ocorrerá com os sete algoritmos mencionados e utilizando dados do *spotify* como base para aplicação. Além disso, também será produzido um tutorial que simplificará este trabalho e será disponibilizado em formato html e o script pelo link do *github* que consta nas considerações finais. Vale mencionar, que técnicas de agrupamento também são conhecidas pelo termo Clusterização, essa palavra tem origem do inglês ‘*Cluster*’ que significa grupo e é

interessante se habituar ao termo, visto que ele é recorrente na literatura e entre os usuários dessas técnicas. Uma perspectiva prática e clara quando se é abordado o tópico de clusterização é explicitar que existem clusterizações hierárquicas e não-hierárquicas, e nessas estrutura temos elementos que serão mencionados nas próximas seções, o desenvolvimento desses termos se dá na tentativa de se encontrar um método viável para particionar elementos.

3. REFERENCIAL TEÓRICO

Diversos autores desenvolveram estudos referentes à agrupamento, como Fisher, Forgy, Macqueen e entre outros. Richard A. Johnson e Dean W. Wicher (2008) contém o tópico relacionado a clusterização que se encontra no capítulo 12 do livro, introduzindo a ideia de agrupamentos, a forma como é abordado o assunto consegue de forma satisfatória tratar de quase todas as áreas ligadas à agrupamento, lidando com conceitos de distâncias, clusterização hierárquica, clusterização não-hierárquica, e outros assuntos do tema, vale também mencionar o capítulo 6 relacionado a comparação de vetores de médias visto que será abordado neste trabalho para uma etapa após a clusterização.

J. MacQUEEN (1967) popularizou uma das técnicas mais comuns de agrupamento Não-hierárquico e também tornou o termo *k-means* conhecido na área de multivariada, em seu trabalho é explicado o comportamento assintótico do processo pela teoria e com aplicações em amostras reais, sendo elas com grupos de alunos, documentos e outros tipos fontes de dados, ele comenta ainda que na tentativa de se encontrar esse método viável de particionamento, Cox (1957) em Note on grouping, Fisher (1958) em On grouping for maximum homogeneity, Edward Forgy (1965) em Cluster analysis of multivariate data: efficiency vs. interpretability of classifications propuseram soluções específicas para algumas situações dentro dessa problemática de agrupamento de dados.

Leonard Kaufman e Peter J. Rousseeuw (1990) aborda clusterização estruturando os pilares necessários para a construção dos *clusters* e com o objetivo de implementar algoritmos de agrupamento na linguagem de Fortran, que realizam o particionamento em torno de

centróides, análises e formas aglomerativas para estabelecer hierarquias entre os dados. O Artigo da Anja Struyf, Mia Hubert e Peter J. Rousseeuw, utilizam como base o livro para a implementação na linguagem de S-PLUS com o mesmo objetivo.

4. BANCO DE DADOS

O banco de dados utilizado para aplicação deste trabalho será extraído da *API* do *spotify*, pelo site *spotify* for Developers. Em que é possível coletar diversos tipos de dados sobre músicas e suas características, foram selecionadas 50 *playlists* de diferentes gêneros musicais que são recomendadas pelo próprio aplicativo. Levou-se em consideração os gêneros que tinha o campo de busca de *playlists* populares. A escolha de cada *playlist* foi por conveniência com o critério de que no mínimo 1 milhão de ouvintes deveriam ter adicionado a *playlist* a sua galeria. No total das 50 *playlists* foram obtidas 5634 músicas e 61 variáveis. O link para o *github* com a planilha que contém a lista de todas as *playlists* está nas considerações finais em um link para o *github* para casos de reaplicação das técnicas desse trabalho. Porém as *playlists* também podem ser atualizadas com frequência, a título de conseguir replicar o estudo se encontrará também uma planilha, em que foi salvo as planilhas utilizadas para os resultados da seção de análise exploratória e o pacote Cluster. Essa coleta foi realizada dentro do ambiente do RStudio e será detalhada no tópico a seguir, ela pode ser utilizada com outras *playlist*:

4.1 *Web scraping*

Ao acessar o site *spotify* for Developers e efetuar o login para criação do seu dashboard. Nesse mesmo ambiente pode ser localizado dois códigos importantes para a realização do *web scraping*, o primeiro é o Client ID e o segundo é o Client Secret. Esses dois códigos serão utilizados pela biblioteca de nome '*spotifyr*'. Após a instalar a biblioteca, teremos vários tipos de funções, mas a função que será utilizada será a: '*get_playlist_audio_features*' a estrutura dessa função é da seguinte forma:

```
get_playlist_audio_features (username, playlist_uris, authorization  
                             =get_spotify_access_token ())
```

em que:

Username: String do nome do usuário do *spotify*. Pode ser encontrado no aplicativo.

playlist_uris: Vetor de elementos com o código de compartilhamento da *playlist*.

authorization: é um campo obrigatório, o token são os códigos mencionados anteriormente.

Com essas informações se torna possível realizar o *web scraping*. Vale ressaltar que existem outras funções e linguagens que podem ser exploradas para esse tipo de coleta e que podem oferecer melhores resultados de acordo com o objetivo do programador, entretanto, as bibliotecas *spotifyr* e a função mencionada conseguem suprir de maneira satisfatória a necessidade para a aplicação dos códigos desse trabalho. O código será dividido em 4 partes e estará no Apêndice A.

4.2 SELEÇÃO DE VARIÁVEIS

A função retorna 61 variáveis, sendo 35 variáveis caracteres, 14 variáveis numéricas, 7 variáveis lógicas, 2 objetos especiais, sendo bancos de descrição do artista e 3 do tipo listas, boa parte das variáveis do tipo caracter e lógica são códigos identificadores, por exemplo temos códigos identificadores de cada música, url da *playlist*, url da foto da *playlist* e outras características que não são interessantes explorar para a aplicação deste trabalho, as variáveis que serão escolhidas estão no Quadro 1 abaixo.

Quadro 1 - Lista de variáveis selecionadas para compor aplicação

nº da var.	Nome da Var.	Tipo	Nome transformado	Descrição
1	Playlist_name	Chr	playlist	Nome da playlist
2	danceability	Num	dançabilidade	Variável numérica que mede o quão dançante uma música em um intervalo de 0 a 1, não é especificado como é medido.
3	energy	Num	energia	Assim como Dançabilidade, mede-se a energia de um música em um intervalo de 0 a 1
4	track.explicit	Lógica	explícito	Conteúdo é classificado como inapropriado
5	loudness	Num	volume	Variável numérica que mede a altura alcançada pela faixa musical
6	mode	Num	modo	Variável que informa se o modo da escala é maior ou menor
7	speechiness	Num	cantado	Variável que mede o quanto se é cantado na música
8	acousticness	Num	acústica	Variável que mede a acústica presente na música
9	instrumentalness	Num	instrumentabilidade	Variável que mede o quão presente os instrumentos estão na música
10	liveness	Num	vivacidade	Variável que mede a vivacidade da música
11	valence	Num	valencia	Variável que mede a coesão/construção rítmica da música
12	tempo	Num	bpm	velocidade de batidas por minuto
13	time_signature	Num	compasso	Fórmula do compasso
14	track.duration_ms	Num	duracao_ms	Tempo de duração da música em milissegundos
15	track.name	Chr	música	Nome da música
16	track.popularity	Num	popularidade	Nível de popularidade da música baseado em acessos
17	track.album.album_type	Fct	tipo	Identifica se a música está presente em um álbum, coletânea ou é um single
18	track.album.name	Chr	álbum	Nome do Álbum
19	key_name	Fct	tom	Tom da música dentro da escala temperada
20	mode_name	Fct	modo_do_tom	Identifica se o tom presente na música é maior ou menor
21	artista	Chr	artista	Nome do artista

Ocorreram dados faltantes na coleta, A função retornou duas linhas vazias. E colocando em vista o objetivo deste trabalho, compreendendo que não haverá impacto na aplicação das ferramentas, foi decidido que serão excluídas. E por último para organizar vamos renomear as variáveis de acordo com a coluna ‘Nome transformado’ do quadro anterior.

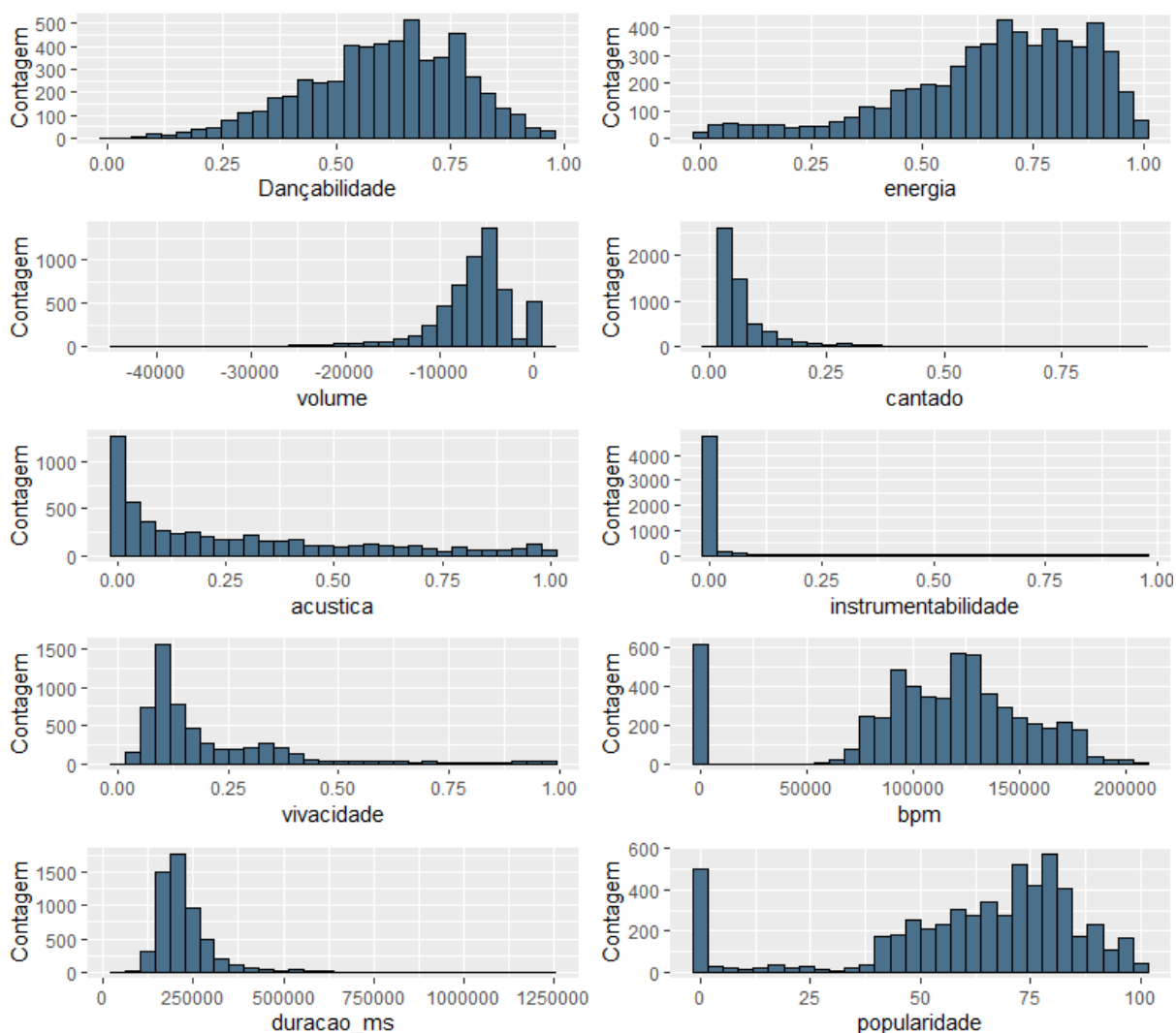
5. ANÁLISE EXPLORATÓRIA

Antes de ir para as ferramentas de fato, é necessário compreender a natureza dos dados, suas amplitudes, possíveis medidas de posição e dispersão, distribuições e tabulações. A primeira consulta será em relação às amplitudes das variáveis numéricas. na Tabela 2, podemos observar as amplitudes e algumas medidas a variável dançabilidade, energia, cantado, acústica, instrumentabilidade, vivacidade tem suas observações contidas entre 0 e 1. A média está abaixo da mediana no caso da dançabilidade e energia. Para a variável cantado, acústica, instrumentabilidade e vivacidade a média é maior que a mediana. As variáveis bpm, duracao_ms e popularidade são maiores que 0. No caso da variável volume a amplitude contém valores negativos e positivos, sendo mais de 75% dos dados negativos. No caso da variável bpm temos 75% acima de 92.033 bpm's. Popularidade oscila entre 0 a 100 e a mediana é maior que a média. Uma variável que chama atenção é a instrumentabilidade, que até 75% dos dados a variável não ultrapassa 0.00124 e o máximo é 0.964, e a mediana é um valor muito próximo de 0. Na Figura 2 é possível analisar as distribuições da densidade dessas variáveis, a Dançabilidade, Energia, Volume e popularidade. Elas apresentam assimetria à esquerda. Bpm e popularidade concentram observações no final da cauda à esquerda. No caso de cantado, acústica, vivacidade, Duracao_ms são assimétricas à direita. Como comentado a variável instrumentabilidade apresenta a maioria das observações, são ou estão próximas a zero. É interessante conhecer também as variâncias e seus respectivos desvios padrões das variáveis. Visto que algumas distâncias podem variar dependendo de grandes diferenças, essas informações estão na tabela 3. As variâncias das variáveis que têm suas amplitudes entre 0 a 1 tem oscilações dentro dos seus intervalos, as outras variáveis apresentam valores altos para suas variâncias, o mesmo vale para os desvios padrões. Visto que as análises apresentam diferenças entre amplitudes e assimetrias, seria interessante padronizar os dados para ajustá-los.

Tabela 1 - Sumarização das variáveis numéricas

Nomes	Mínimo	Q1	Mediana	Média	Q3	Máximo	Variância	Desvio padrão
Dançabilidade	0.000	0.494	0.619	0.6027	0.732	0.965	0.028	0.169
energia	0.002	0.549	0.699	0.664	0.829	0.996	0.047	0.217
volume	-43.738	-8.118	-5.731	-6.572	-4.053	1.906	22383535.390	4731.12
cantado	0.000	0.036	0.050	0.079	0.086	0.922	0.006	0.078
acústica	0.000	0.024	0.174	0.275	0.451	0.996	0.083	0.288
instrument.	0.000	0.000	0.000	0.063	0.001	0.964	0.039	0.198
vivacidade	0.013	0.093	0.132	0.201	0.260	0.989	0.030	0.174
bpm	0.000	92.033	118.384	109.897	138.162	207.265	2214297936.210	47056.320
duracao_ms	58.149	175.493	206.710	225.888	250.626	1.252.321	7489680709.210	86542.940
popularidade	0.000	51.000	69.000	61.730	79.000	100.00	657.437	25.640530

Figura 1 - Histograma das variáveis numéricas do banco de dados extraído do *spotify*



Com a compreensão de como estão distribuídos os valores das variáveis, pode-se averiguar as relações entre as variáveis. Podemos visualizar pelo mapa de calor das correlações na Figura 3 em que a cor avermelhada representa correlações positivas, as correlações negativas estão representadas pela cor azulada. No geral as correlações bem fracas, algumas podem ser classificadas como fracas por ter uma correlação um pouco mais elevada. As mais altas que temos são classificadas como moderadas sendo elas: a única positiva obtida o volume e a energia e as correlações negativas que são a energia e acústica, energia e instrumentabilidade, volume e instrumentabilidade. Temos então indícios que a maioria das variáveis provavelmente são não correlacionadas. Podemos observar pelos gráficos de dispersão na Figura 4 como elas se relacionam. Os gráficos 2x2 com todos os pares possíveis entre as variáveis numéricas mostram que boa parte das massas de dados não apresenta uma relação linear. Alguns pares apresentam comportamentos quadráticos, cúbicos ou exponenciais, de qualquer forma a baixa correlação linear dá indícios que as variáveis não são dependentes entre si. Outra função interessante que retorna vários é da biblioteca GGally, a função ggpairs retorna todos os possíveis gráficos 2x2. Um teste associado à correlação é a densidade da variável. No gráfico da Figura 5, os gráficos de dispersão são iguais aos da Figura 4, porém vale mencionar os testes realizados na matriz triangular superior. É medido a sua correlação utilizando os coeficientes do momento do produto de Pearson, o tau de Kendall ou o Rho de Spearman. Os três testes estimam a associação entre amostras emparelhadas e se o valor é igual a 0. A escolha de qual método será utilizado depende se a amostra segue distribuição normal bivariada e o seu tamanho. Se for normal bivariada, será utilizado o teste de Pearson, seguindo distribuição t ($n-2$) graus de liberdade, n sendo o comprimento do vetor x . Caso não siga normal bivariada, pode se utilizar o teste de Kendall ou o teste de Spearman. O teste de Kendall é calculado se houve pelos menos 50 amostras emparelhadas, para $n < 1290$ será aplicado o teste de Spearman, se for maior será realizada uma aproximação t assintótica. Os símbolos “*”, “**”, “***” e “.” representam a significância. Sendo elas o valor da probabilidade (p) da estatística de teste menor que: $p < 0.05$, $p < 0.01$, $p < 0.001$, $p < 0.1$ respectivamente. No geral no gráfico 5, ao nível de significância de 5% rejeita-se a hipótese de que o valor é igual a zero.

Figura 2 - Heatmap com as correlações das variáveis numéricas do banco de dados extraído do *spotify*

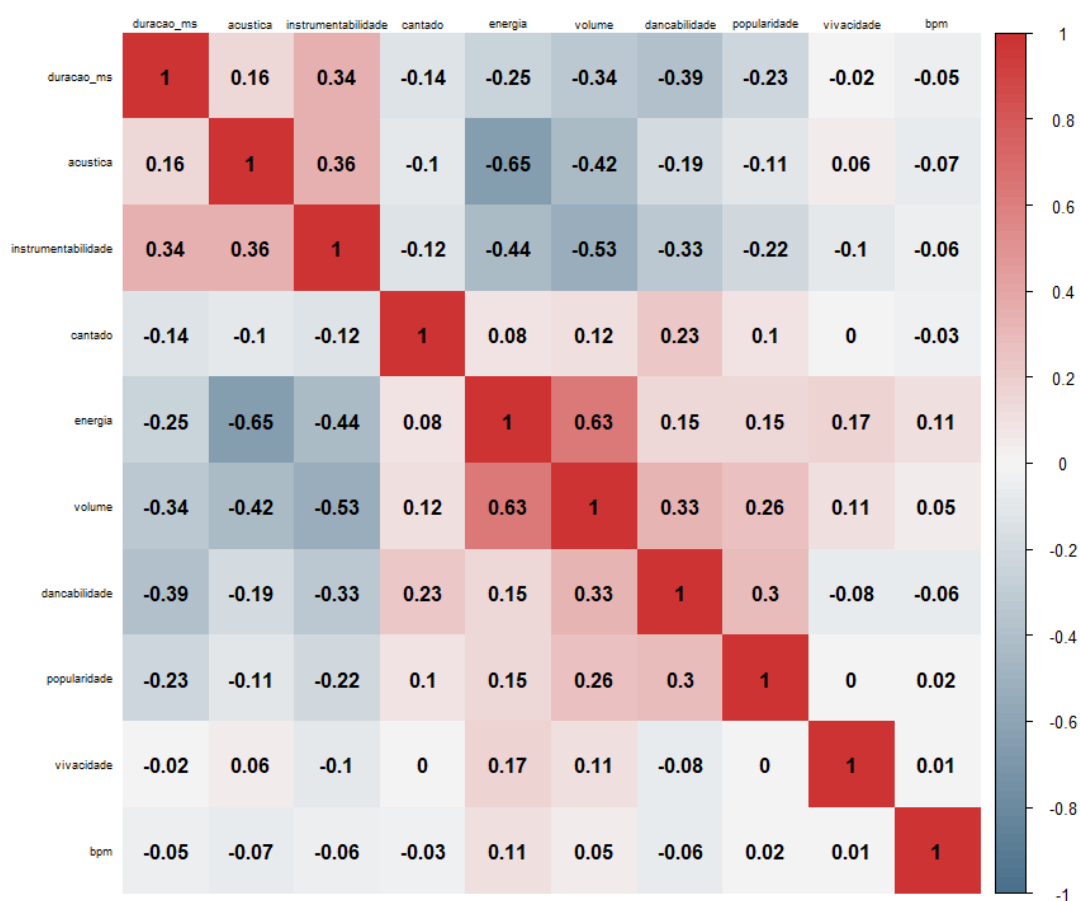
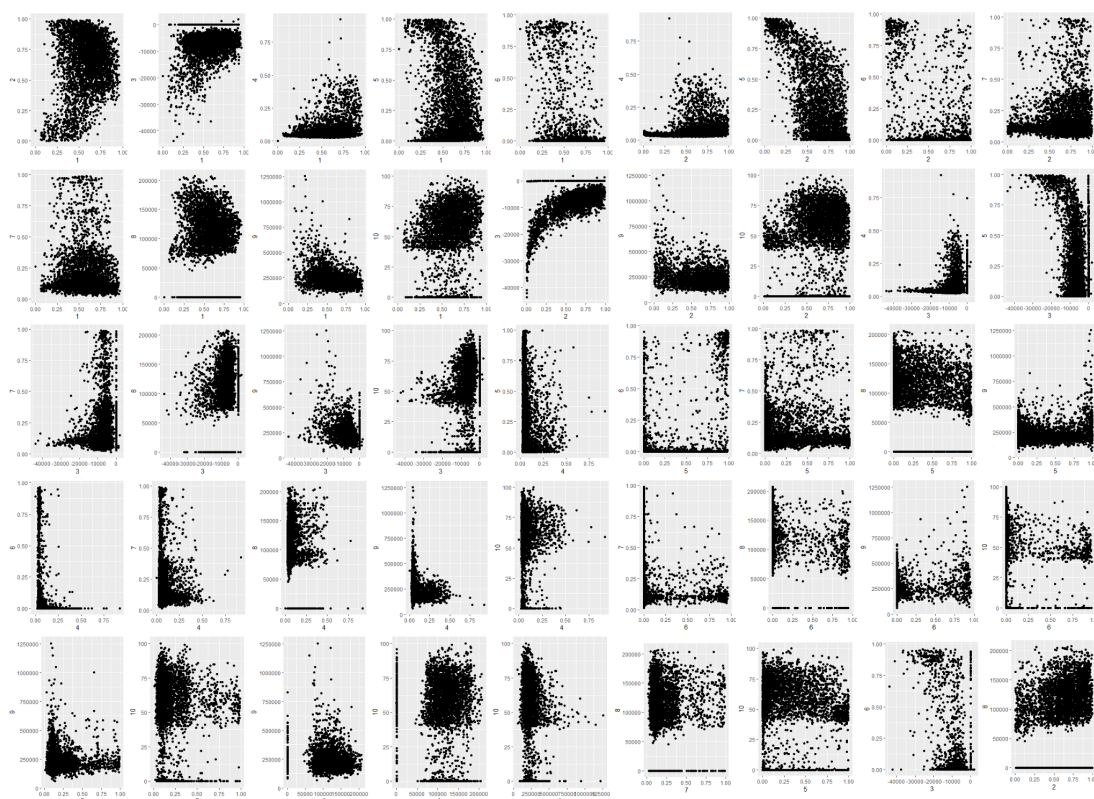
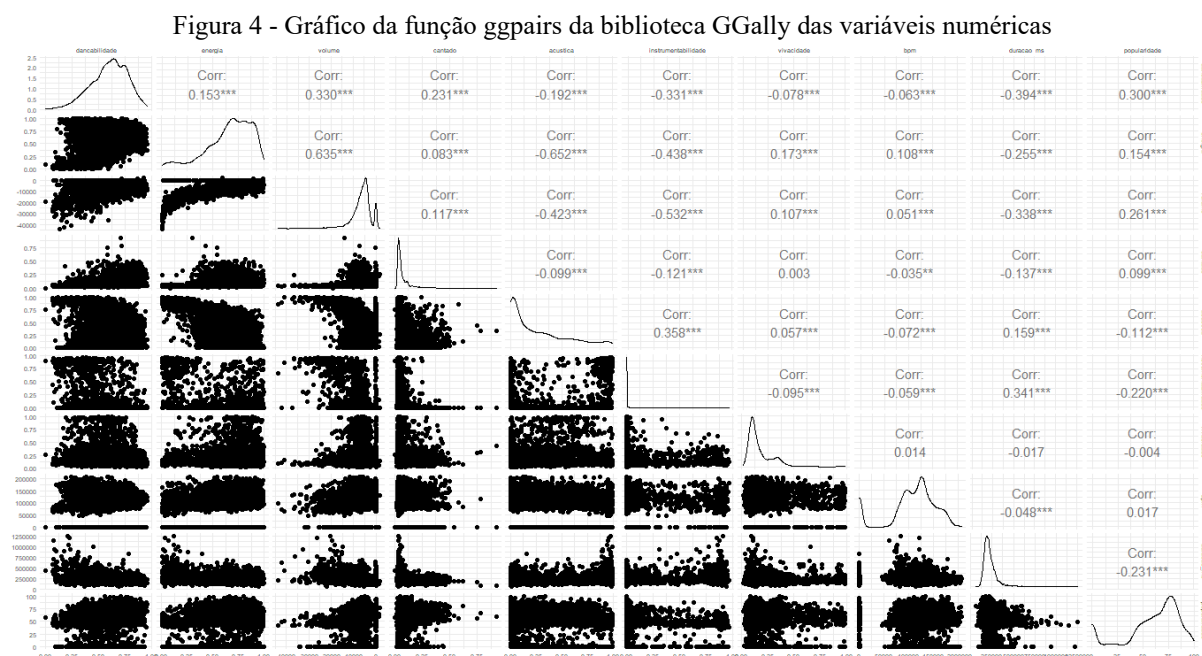


Figura 3 - Gráficos de Dispersão 2x2 das variáveis numéricas do banco de dados extraído do *spotify*



1 = Dançabilidade - 2 = Energia - 3 = Volume - 4 = Cantado - 5 = Acustica - 6 = Instrumentabilidade - 7 = Vivacidade - 8 = Bpm - 9 = duracao_ms - 10 = popularidade



Avaliando algumas variáveis de origem não numéricas. Primeiro podemos verificar na Figura 6 os tons mais recorrentes, o C# (dó sustenido) é o tom mais frequente com 652 músicas e o menos frequente é D# (ré sustenido) com 238. Na Figura 7 temos 2963 músicas que pertencem a álbuns, enquanto temos 2405 singles e 265 músicas independentes. Na Figura 8 temos os artistas que mais aparecem no data frame, sendo eles *Måneskin* com 61 músicas, seguidos por João Gomes e *AC/DC* com 44 músicas. Ao todo são 1334 artistas, o gráfico só demonstra os artistas que tiveram mais de 20 músicas. Na Figura 9 temos a proporção de músicas que apresentam conteúdo classificado com explícito, sendo 82.71% classificado como conteúdo não explícito.

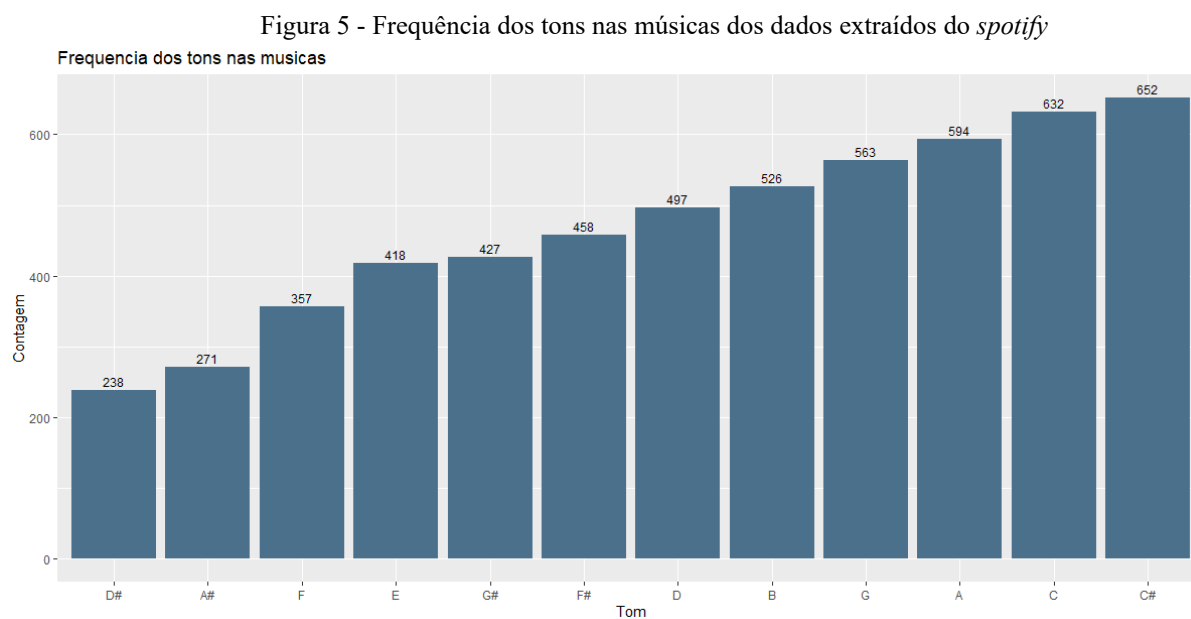


Figura 6 - Frequência dos tipos de organização de músicas no banco de dados

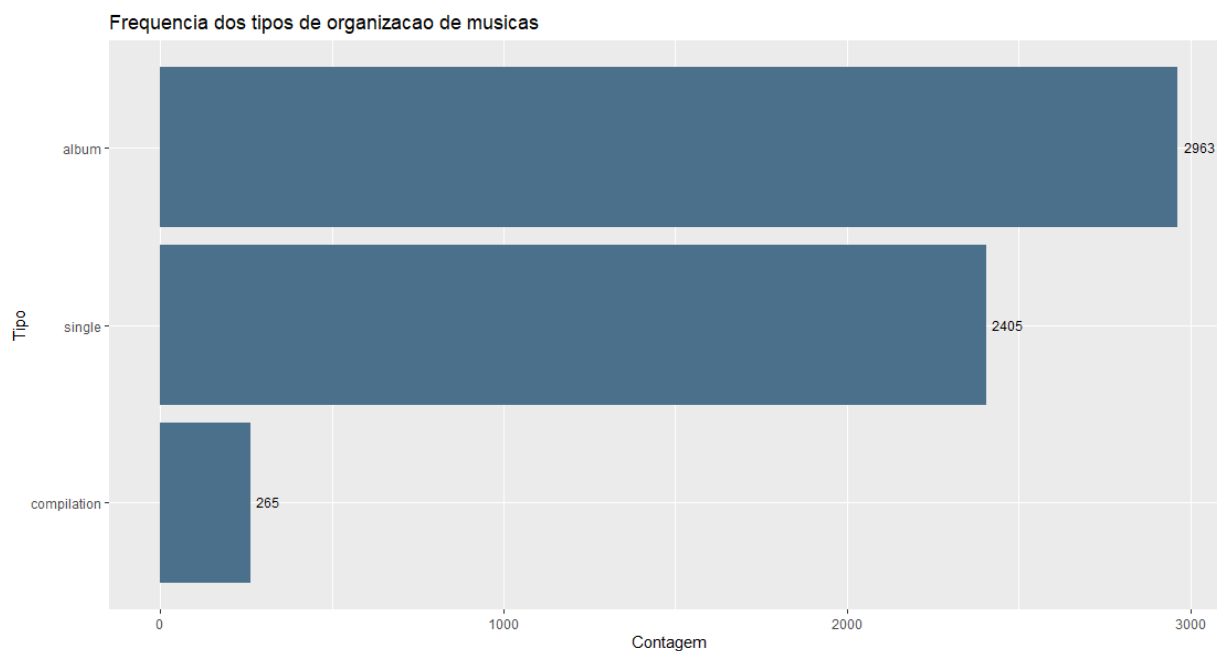


Figura 7 - Artistas mais frequentes no banco de dados com mais de 20 músicas presentes

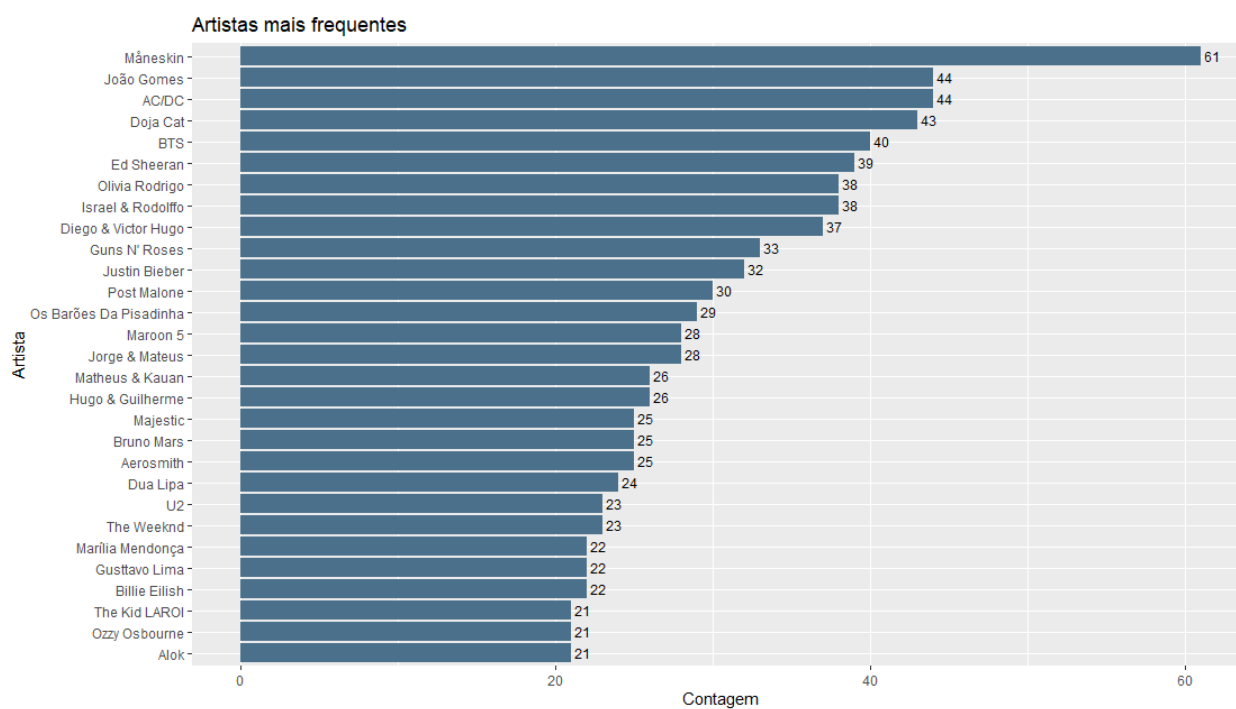
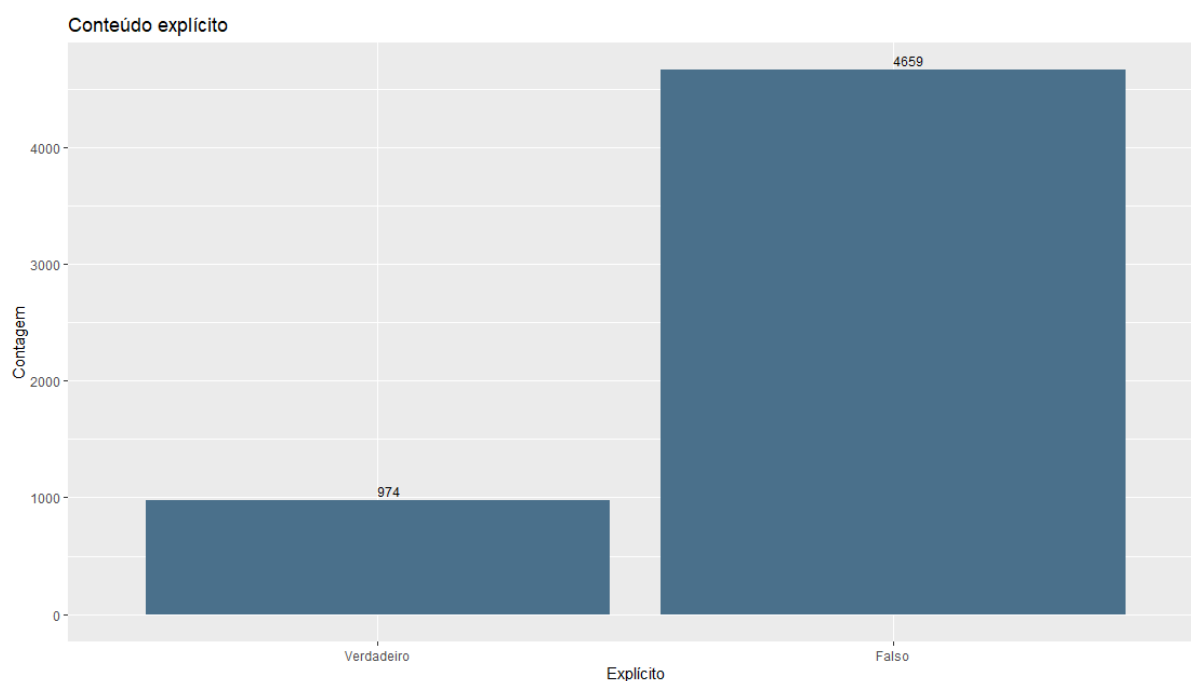


Figura 8 - Quantidade de conteúdo explícito no banco de dados



6. MÉTODOS

É importante para compreensão desse assunto entender as etapas que precedem a clusterização. Então como realizado na seção anterior, a análise exploratória nos dá as informações que estão disponíveis para então estabelecermos um objetivo. Ele pode ser formado em decorrência da análise exploratória ou por conhecimento empírico do pesquisador. Estabelecido que tipo objetivo está será almejado, podemos determinar qual técnica de clusterização poderá corresponder da melhor forma possível. O estudo desses métodos partem de alguns conceitos (Distâncias, Matrizes e outros) que estruturam as funções de clusterização. O agrupamento utiliza tipos de distâncias e/ou matrizes para realizar se a classificação será para o *cluster* 'A' ou para o *cluster* 'B', ou se deverá agrupar ou dividir suas observações. Isso se dá pela característica da técnica utilizada, foi criado na página seguinte para a compreensão desse assunto um fluxograma na Figura 10 que pode ser usado de guia para aplicação das técnicas de agrupamento.

6.1 Métodos Hierárquicas:

Nesse tipo de método não se foca em um número exato de grupos mas na altura em que se deve analisar para a formação do *cluster*. A construção se dá partindo de um *cluster* maior e separando as observações em *clusters* menores ou cada observação é um *cluster* e nos próximos passos se agrupa em grupos maiores. O critério para esses agrupamentos variam de acordo com a técnica. No geral, esses algoritmos são conhecidos como métodos aglomerativos e divisivos.

6.2 Métodos Não-hierárquicos:

O foco dos métodos Não-hierárquicos no geral, é encontrar o número “k” de *clusters* que consiga realizar a divisão das observações de maneira satisfatória, ou seja, que consiga identificar semelhanças e diferenças entre as observações.

Esses métodos se baseiam em sua maioria na densidade das observações e nas relações entre as variáveis. Os métodos hierárquicos têm a vantagem de que sua construção se dá criando vários valores para “k” e com isso é possível ter uma visão ampla das observações. Em contrapartida, sua qualidade também se demonstra um defeito, visto que após executada uma etapa da aglomeração ou divisão isso não poderá ser mudado em etapas subsequentes. No caso dos Não-hierárquicos a problemática se dá pelo número de grupos. Porém com os métodos de agrupamento, a questão se dá pela subjetividade dos seus resultados. É importante ter em mente que, por mais que o resultado aparente seja significativo, na realidade está nos dando indícios das características dos dados. Então esses métodos devem ser utilizados para formação de possíveis hipóteses e análises em relação a grupos e perfis.

Fluxograma para clusterização

GUIA DE ETAPAS E TOMADA DECISÃO

NECESSIDADE DE CLUSTERIZAR

Por análise exploratória ou conhecimento prévio do pesquisador é estabelecido o objetivo de checar a existência de possíveis grupos

QUAL MÉTODO DEVE SER USADO

Que tipo de dados estão disponíveis?

(Numéricos, binários, fatores nominais/ordinais...)

Com base nos dados que tipo de agrupamento pode ser feito? particionado em k grupos ou múltiplos níveis de clusters?

Existem muitas variáveis? é necessário reduzir dimensionalidade?

CLUSTERIZAÇÃO HIERÁRQUICA

CLUSTERIZAÇÃO NÃO HIERÁRQUICA

MÉTODO DIVISIVO

MÉTODO AGLOMERATIVO

MÉTODO DE PARTICIONAMENTO

O Método necessita de calculo de métrica?

Sim

APLICAÇÃO DE MÉTRICAS TIPOS DE DISTÂNCIAS

com base na análise exploratória determinar melhor distância

FORMAÇÃO DA MATRIZ DE DISSIMILARIDADE

DETERMINAÇÃO DO Nº DE GRUPOS E ALTURA

Gráfico do cotovelo, Gráfico da Silhueta ou conhecimento prévio do nº de grupos

HIERÁRQUICA

NÃO HIERÁRQUICA

APLICAÇÃO DA FUNÇÃO

Algoritmos como: Divisive Analysis Clustering, Agglomerative Nesting, hclust...

APLICAÇÃO DA FUNÇÃO

Algoritmos como: Kmeans, DBscans, Partitioning Around Medoids, Fuzzy Analysis...

GRÁFICO DE DISPERSÃO

aconselhável para clusterização com até 3 variáveis

TABELA COM MEDIDAS DOS GRUPOS

Medidas das médias dos grupos por atributo, sua quantidades e variâncias

FORMAÇÃO/DESCRIÇÃO DO PERFIL DE CADA CLUSTER

é interessante comparar as médias e variâncias dos grupos.

AJUSTES?

Caso não tenham sido formados grupos satisfatórios para o objetivo, é interessante testar outros métodos e métricas voltando em uma das etapas e ajustando o método

FIM

Vale ressaltar que nem sempre será possível formar bons grupos, mas isso se dará pelos objetivos e variáveis disponíveis.

6.3 Distâncias

O cálculo das distâncias é o item primordial para a clusterização, pois os métodos de agrupamento, em geral, se desenvolvem a partir delas. Considere um ponto no plano cartesiano com uma coordenada $p_1=(x_1,x_2)$ e estabelecendo o ponto de origem sendo a coordenada $O=(0,0)$. Denota-se $d(O,p_1)$ a distância entre o ponto e a origem, essa distância pelo teorema de pitágoras é igual a expressão (1). Ao generalizar essa fórmula para o espaço vetorial de dimensão n , ou seja, sairmos do plano no $\mathbb{R}^2$ para o $\mathbb{R}^n$, agora com $p_1 = (x_1, x_2, \dots, x_p)$, a expressão se torna a (2). É possível medir a distância entre pontos, no caso com essa generalização podemos substituir o ponto de origem por outro vetor coordenada, então seja $P_2 = (y_1, y_2, \dots, y_n)$ a distância entre os pontos se torna a (3).

$$d(O, p_1) = \sqrt{x_1^2 + x_2^2} \quad (1)$$

$$d(O, p_1) = \sqrt{x_1^2 + x_2^2 + x_3^2 + \dots + x_p^2} \quad (2)$$

6.3.1 Distância Euclidiana

A expressão (3), calcula a distância em linha reta entre pontos. Essa fórmula que é generalizada do teorema de pitágoras também é conhecida como distância euclidiana. É importante perceber que cada medida tem uma contribuição igual na fórmula. E isso pode ocasionar problemas dependendo das grandezas. Com isso é possível utilizar de outras medidas de distâncias, que façam ponderações e/ou ajustes dependendo da situação das variáveis.

$$d(p_1, p_2) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + \dots + (x_p - y_p)^2} \quad (3)$$

6.3.2 Distância Manhattan

Outra distância muito utilizada é a de Manhattan, a fórmula dessa distância se dá pela expressão (4), quando Kaufman e Rousseeuw(1990) citam o contexto em que essa distância foi aplicada.

“A distância de Manhattan foi usada em uma análise de *cluster* contexto por Carmichael e Sneath (1969) e deve seu nome peculiar ao seguinte raciocínio. Suponha que você more em uma cidade onde as ruas são todas norte-sul ou leste-oeste e, portanto, perpendiculares entre si, as ruas são retratadas como linhas verticais e horizontais. Então, a distância real que seria necessário para viajar de carro para ir do local “i” ao local “j” totaliza $|x_{i1} - x_{j1}| + |x_{i2} - x_{j2}|$ correspondendo a (4). Este seria o mais curto comprimento entre todos os caminhos possíveis de “i” a “j”. Só um pássaro pode voar reto do ponto “i” ao ponto “j” cobrindo assim a distância euclidiana entre esses pontos.” (Kaufman e Rousseeuw, 1990, p.12, tradução nossa)

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + |x_{i3} - x_{j3}| + \dots + |x_{ip} - x_{jp}| \quad (4)$$

Distâncias preservam as seguintes características:

- i) $d(i, j) \geq 0, \forall i, j \in \Omega$
- ii) $d(i, i) = 0, \forall i, j \in \Omega$
- iii) $d(i, j) = d(j, i), \forall i, j \in \Omega$
- iv) $d(i, j) \leq d(i, h) + d(h, j), \forall i, j \in \Omega$

Em que h é um terceiro ponto que forma um par de dissimilaridade e que Ω representa todos valores possíveis nos dados. Essas características estabelecem que as distâncias sejam positivas. A distância entre a observação e ela mesma é zero. A distância de i para j é igual à

distância de j para i e a distância de i e j é menor ou igual que a soma das distâncias de i e h mais a distância de h e j. outras formas de se calcular a distância que estão disponíveis no pacote *cluster* ou são popularmente conhecidas são:

6.3.3 Gower:

$$d(i, j) = \frac{\sum_{f=1}^p \delta_{ij}(f) d_{ij}(f)}{\sum_{f=1}^p \delta_{ij}(f)} \quad (5)$$

A Métrica é descrita por Leonard Kaufman e Peter J. Rousseeuw (1990) utilizando para tratamento de variáveis mistas. Nessa métrica $\delta_{ij}(f)$ é igual a 1 se não houver dado faltante na k-ésima variável e será zero caso contrário. $\delta_{ij}(f)$ também será zero quando a variável é assimétrica ou os atributos binários correspondendo a 0-0. A função não pode ser calculada se o vetor for todo de zeros, no caso do $d_{ij}(f)$ é a dissimilaridade de i e j na j-ésima variável, no caso de variáveis nominais ou binárias simétricas. A expressão (5) representará o número de correspondências sobre o número total de pares disponíveis, sendo igual a (6). Em que p é número total de variáveis, u é o número de correspondências. No caso da variável ser binária assimétrica é utilizado o coeficiente de Jaccard. Para o caso da variável ser intervalar é utilizada a expressão (7). O programa irá explorar essas expressões combinando na fórmula do (8), em que w_1, w_2, w_3 são pesos e se referem às três matrizes resultantes possíveis.

$$s(i, j) = \frac{u}{p}, \quad d(i, j) = \frac{p-u}{p} \quad (6)$$

$$d_{ij}(f) = \frac{|x_{if} - x_{jf}|}{R_f}, \quad R_f = \max_h x_{hf} - \min_h x_{hf} \quad (7)$$

$$d(i, j) = \frac{w_1 d_1(i, j) + w_2 d_2(i, j) + w_3 d_3(i, j)}{w_1 + w_2 + w_3} \quad (8)$$

6.3.4 Coeficiente de Jaccard:

$$J(A, B) = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \quad (9)$$

O coeficiente de Jaccard mede a similaridade entre conjuntos, em que $|A \cap B|$ é o valor absoluto da interseção dos conjuntos A e B. Partindo do seu coeficiente é possível construir a distância de Jaccard pelo complementar da expressão (9), ou seja, $d_j(A, B) = 1 - J(A, B)$.

6.3.5 Squared Euclidean:

$$d^2(p1, p2) = (x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + \dots + (x_p - y_p)^2 \quad (10)$$

Essa distância (10) é oriunda da distância euclidiana, porém elevada ao quadrado, uma função dentro do pacote *cluster* tem essa distância disponível para uso, ele é útil para comparações e em métodos estatísticos.

6.3.6 Minkowski:

$$d(i, j) = (|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + |x_{i3} - x_{j3}|^q + \dots + |x_{ip} - x_{jp}|^q)^{\frac{1}{q}} \quad (11)$$

Essa distância (11) é uma generalização da construção realizada para as distâncias euclidiana e manhattan, note que, se $q=2$ a expressão se torna a distância Euclidiana e se $q=1$ temos a distância de Manhattan.

6.3.7 Canberra:

$$d(x,y)=\sum_{i=1}^n \frac{|x_i-y_i|}{|x_i|+|y_i|} \quad (12)$$

A distância de Canberra (12) é uma medida ponderada da distância de Manhattan.

6.4 Matriz de Dissimilaridade

A matriz de Dissimilaridade é uma matriz $n \times n$ que pode também ser expressa com uma matriz triangular inferior ou superior, em que suas entradas são os tipos de distâncias apresentadas nos tópicos anteriores. A dissimilaridade ou os coeficientes de dissimilaridade são números não negativos, que preservam características para os pares de observações. $d(i, j)$ (lê-se: a dissimilaridade do par i e j , ou distância de i e j), são pequenos, próximos de zero quando i e j estão perto um do outro e $d(i, j)$ é grande quando i e j são muito distantes um do outro. Normalmente se assume que essas diferenças são simétricas e que o par $d(i, i)$ é igual a zero, no geral ela satisfaz as primeiras propriedades mencionadas anteriormente na seção manhattan i), ii), iii) e não é obrigatório o cumprimento iv).

6.5 k-means

Um dos métodos mais comuns dentro da clusterização é o método k-means. O termo foi popularizado pelo artigo *Some Methods for Classification and Analysis of Multivariate Observations* do J. MacQUEEN (1967). Nele relata-se a dificuldade de se encontrar métodos de agrupamentos e propõem uma maneira recursiva para particionar os dados. Sendo de simples aplicação e rápido processamento, a lógica do algoritmo segue 3 passos:

-
1. Escolhido o número de grupos, denominado k
 2. Dentro dos dados atribui-se a alguma observação aleatória a um cluster, depois utilizando alguma medida de distância, se atribui ao elemento mais próximo o mesmo *cluster* e calcula-se a média das distâncias, formando o centro do cluster.
 3. repete-se o passo 2, e se recalcula o centro do *cluster* dado o novo objeto que entrou, isso se repete até todos os dados tenham seus respectivos clusters.

A grande questão é a quantidade de grupos que devem ser utilizados para os dados. Principalmente se não houver um conhecimento prévio/empírico sobre os dados em questão. Por isso, é importante após realizarmos a clusterização não-hierárquica avaliar os *clusters* que foram obtidos, existem maneiras dessas avaliações, e também meios de dar um número para o cálculo do k ideal.

6.6 Silhueta

Uma maneira de calcular o número de grupos necessários ou a altura de divisão dos *clusters* na clusterização hierárquica, é pelo método da silhueta. Normalmente retorna-se um gráfico que expressa o número ideal de partições. Isso se dá pois o objetivo do método da silhueta é calcular uma medida que expresse o quão parecida a observação é em relação ao cluster. Porém, note que nessa etapa nenhuma técnica foi aplicada, nenhum *cluster* foi gerado. A silhueta tem a fórmula (13) que depende diretamente da (14) e (15), essas duas expressões tem como objetivo calcular a coesão das observações em seus *clusters* (14) e a divisão entre os *clusters* (15). Esse coeficiente utiliza o valor k que é número de *clusters* que está sendo utilizado. k_c é o c -ésimo cluster, e $d(i, j)$ é a distância do par de observações. A variação de $s(i)$ é entre 0 e 1, se $s(i)$ está próximo de 1 implica que $b(i) > a(i)$, ou seja temos que a observação está bem estabelecida no seu respectivo cluster. Se $s(i)$ for igual a -1, significa que o *cluster* deveria pertencer ao *cluster* vizinho. No caso de $s(i)$ igual a 0 significa que a observações está na região das fronteiras entre os grupos, a média dessas medidas informa

quão bem agrupado está o cluster. então pela expressão (16) temos que o máximo valor médio dos valores de $s(i)$ ($\overline{s(i)}$) determina qual o valor k será o ideal para se analisar.

$$s(i) = \frac{b(i)-a(i)}{\max\{a(i);b(i)\}} \quad (13)$$

$$a(i) = \frac{1}{|k_c|-1} \sum_{j \in k_c \text{ e } i \neq j} d(i, j) \quad (14)$$

$$b(i) = \min_{c \neq i} \frac{1}{|k_c|} \sum_{j \in k_c} d(i, j) \quad (15)$$

$$S_k = \max_k \overline{s(i)} \quad (16)$$

6.7 Métodos de avaliação

Após as etapas de construção e aplicação da função escolhida, um momento importante desse processo é a avaliação dos grupos. Explorando a construção do fluxograma, gráficos de dispersão podem ser interessantes para aplicações nos casos em que estamos no espaço vetorial do R^2 , ou seja com duas variáveis. Isso pode ser explorado até três variáveis, e para isso é necessário alguns pacotes específicos na linguagem de programação *R* para visualização em três dimensões. Para o caso de quatro dimensões ou mais temos uma boa chance desse tipo de visualização não ser satisfatória, então o mais recomendado é explorar os *clusters* por tabelas. Em que devem ser expressas características de cada *cluster*, seja o número de observações por grupo ou a média e centróides desses grupos podem ser exploradas amplitudes e assim sucessivamente. Essa análise descritiva do *cluster* pode auxiliar na formação de perfil. No caso hierárquicos, ao determinar a altura que será utilizada

para formar os *clusters*, a utilização de dendrogramas é o método mais utilizado para esse tipo de situação. Após essa primeira avaliação uma estratégia interessante para complementar a formação desses grupos é testar estatisticamente possíveis diferenças dos vetores de médias entre os grupos. Iremos comentar brevemente duas ferramentas que podem ser utilizadas para essa situação, a primeira é a análise multivariada de variâncias (Manova) e posteriormente o teste para comparações de médias ou teste de Tukey.

6.7.1 Manova

A análise multivariada de variâncias ou como é mais conhecida, Manova, é utilizada para investigar se o vetor com as médias das observações são iguais ou se apresentam diferenças significativas entre elas. Para a realização da Manova são necessárias 3 suposições que de acordo com Richard A. Johnson e Dean W. Wichern(2007) podem ser relaxadas se forem utilizados com teorema do limite central. As 3 suposições são: as amostras serem aleatórias com médias provenientes de populações diferentes e independentes; As populações possuem matriz de covariância comum e cada população é uma normal multivariada. Como comentado anteriormente o tópico não tem interesse em se aprofundar neste assunto visto a extensão de seu conteúdo. Mas é importante ressaltar que o teste realizado utiliza a distribuição F e em sua hipótese visa a comparação das médias da sua população. E que caso um par de médias apresentar diferença uma da outra consequentemente o teste recusa a hipótese que as médias são iguais. Essa observação gerou um comentário interessante de Brian Everitt e Torsten Hothorn(2011), afirmando que o ganho dessa técnica não é tão eficiente justamente por causa dessa hipótese, pois após essa diferença recusar a hipótese é necessário testar os pares para identificar qual se demonstra diferente. Note que se houver apenas duas populações e a hipótese for rejeitada saberemos que o único par existente se difere um do outro.

6.7.2 Teste de Tukey

Com base na questão da hipótese da Manova, normalmente se a hipótese é rejeitada é necessário testar cada par. O Teste de Tukey visa realizar a comparação múltipla de médias. Para isso, o teste utiliza três suposições que são: as observações são independentes entre elas e entre os grupos, seguem distribuições normal e há homogeneidade de variâncias. O teste de tukey é muito semelhante ao teste t, sua estatística é:

$$q_s = \frac{y_a - y_b}{SE} \quad (17)$$

Em que se y_a é a maior média e y_b é a menor e SE é o erro padrão das somas das médias. Assim o valor de q_s é um valor da distribuição da t-student. então a hipótese nula é que as médias são iguais estatisticamente e em caso de rejeição da hipótese elas diferem.

7. PACOTE CLUSTER

A partir desta seção será explorado o pacote *cluster*, o guia sobre todas as funções e possíveis aplicabilidades. É importante que seja compreendido que o objetivo da clusterização influencia na escolha da técnica assim como foi visto no fluxograma no início da seção. Então após explorarmos as funções complementares do pacote *cluster*, será abordado os algoritmos comentados no objetivo deste trabalho. Vale um comentário, de que as funções não permitem que seja renomeado os eixos e títulos e por essa razão alguns gráficos estão com palavras de origem inglesa. O Pacote *cluster* tem como objetivo realizar a análise de *cluster* no R sendo uma extensão do artigo da Struyf, et al.(1997) em que adiciona as ferramentas na linguagem de S-PLUS baseado no trabalho do Kaufman e Peter J. Rousseeuw (1990). Assim, o R se torna a terceira linguagem a receber esses algoritmos. Na documentação da biblioteca temos 9

bancos de dados, são eles agriculture, animals, chorSub, flower, plantTraits, pluton, ruspini, votes.repub, xclara e 24 funções. Sendo 7 os algoritmos mencionados na seção de objetivos e outras 17 funções. Não é o foco explorá las, porém vale a menção e em caso de necessidade podem ser úteis para determinadas tarefas, são elas:

1. bannerplot: Visualiza agrupamentos hierárquicos ou uma estruturas de dendrogramas binários.
2. clusGap: Busca calcular uma boa medida para número de grupos, dado a necessidade de se estabelecer um k para particionar, ele tenta comparar o $\log(W(k))$ com $E*\log(W(k))$, sendo essa segunda parte calculada pelo método de bootstrap.
3. clusplot: Gera um gráfico 2x2 utilizando a partição gerada do algoritmo.
4. coef.hclust: Calcula o coeficiente aglomerativo, serve para a função Diana, em que mede a estrutura do agrupamento no conjunto de dados, pode se dizer também que é a largura média ou porcentagem de preenchimento do gráfico banner plot.
5. coefHier: Calcula o coeficiente aglomerativo, porém utilizando o objeto “heights” que é retornado pela função Agnes e a Diana, uma observação deixada é que essa medida não deve ser usada para comparar conjuntos de dados que apresentem grandes diferenças em seus tamanhos.
6. ellipsoidhull: Cria um elipsóide 2D em que engloba todos os pontos, interessante se houve relações lineares entre as duas variáveis.
7. ellipsoidPoints: Calcula um elipsóide 2D com base em pontos específicos.

-
8. `lower.to.upper.tri.inds`: Reordena em matrizes triangulares superiores ou inferiores os vetores que são armazenados de forma contígua.
 9. `maxSE`: Assim como a função `clusGap`, a função `maxSE` é uma das possíveis tentativa de encontrar o k ideal para as particionar, ela leva em consideração as lacunas e o desvio padrão, é possível retornar uma matriz com 5 métodos para esse objetivo, `firstSEmax` (localiza o valor $f(.)$ que não é menor do que o primeiro máximo local), `Tibs2001SEmax` (usa o critério de Tibshirani em que propuseram que o menor k tal que $f(k) \geq f(k+1) - s\{k+1\}$), `globalSEmax` (seguindo supostamente as proposições do Tibshirani procura o valor $f(.)$ que não é menor do que o primeiro máximo local), `firstmax` (retorna o primeiro máximo local encontrado), `globalmax` (retorna o máximo global).
 10. `meanabsdev`: Função de *cluster* interna, é comentado que ela não deve ser utilizada pelos usuários.
 11. `pltree`: Retorna o gráfico do dendrograma para as funções Agnes e Diana.
 12. `predict.ellipsoid`: Calcula pontos no limite da elipsóide.
 13. `Silhouette`: Calcula as informações da silhueta de acordo com um determinado número de *clusters*.
 14. `sizeDiss`: Retorna o número correspondente a uma matriz de dissimilaridade como objeto, ou o número de linhas ou colunas de uma matriz triangular (superior ou inferior) é fornecida.
 15. `sortSilhouette`: Ordena as linhas do gráfico de silhueta.

16. `upper.to.lower.tri.inds`: Reordena em matrizes triangulares superiores ou inferiores os vetores que são armazenados de forma contígua.

17. `Volume`: Calcula o volume do objeto por uma função genérica e possui métodos para elipsóides.

Cada uma dessas funções tem seus parâmetros e características mais específicas. Caso seja de interesse pode-se consultar a documentação utilizando a função `help('nome da função')`. Essas ferramentas servem de complemento para os algoritmos das próximas seções, porém não é obrigatório utilizá-las visto que existem outras ferramentas que podem apresentar resultados de performances e esteticamente melhores. Porém agora vamos às funções dos algoritmos autônomos que servem para realizar a clusterização. Nas próximas seções iremos apresentar as principais funções para clusterização. Ligadas a cada um de seus objetivos. Também será realizada uma aplicação com banco de dados que foi construído nas seções anteriores, explorando as ideias possíveis para cada função, além de explorar os caminhos construídos no fluxograma(10) na seção de métodos.

7.1 DAISY

A primeira etapa ao iniciar o processo de clusterização é calcular a matriz de dissimilaridade, a função na biblioteca `cluster` que realiza essa etapa é chamada de Daisy, apelido dado para o programa Computing Dissimilarities with the Program. Essa função é uma das que mais se diferem das outras pois é a única que realiza essa tarefa de calcular a matriz de dissimilaridade. Ela pode ser utilizada dentro de outras funções, são elas a Pam, Agnes, Diana e a Fanny, ficando de fora a Mona e a Clara. Iremos entender essa inclusão na seção de cada uma delas, a grande vantagem dessa função é conseguir trabalhar com variáveis mistas, ou seja, as variáveis podem ser do tipo escala de intervalos, numéricas, fatores nominais ou ordinais.

```
daisy(x, metric = tipo de distancia ("euclidean", "manhattan", "gower"),  
      stand = FALSE, type = list(), weights = rep.int(1, p))
```

A função solicita um argumento obrigatório que é nomeado de **x**, em que **x** que é a matriz ou o data frame com as observações. As dissimilaridades são feitas entre as linhas de cada variável. A função comenta que outros tipos de fontes de variáveis devem ser especificadas, além disso vale especificar que variáveis tipo caractere não podem ser aplicadas. É recomendável com parcimônia, que caso seja de interesse usá-las, é possível transformar a variável em fator, mas isso em casos específicos, em que a variável possa e faça sentido virar um fator. Os outros argumentos são ditos como não obrigatórios, são eles: **metric**, em que recebe a distância que será utilizada, as atuais são: euclidiana (padrão), manhattan e a gower, quando a matriz é de variáveis mistas independente da métrica que é passada, a função utiliza a métrica gower que é a apropriada para essas situações. O terceiro parâmetro existente para a Daisy é o **stand**, a variável recebe o valor lógico TRUE ou FALSE, no caso de receber o valor TRUE a matriz x é padronizada para cada coluna antes de calcular as dissimilaridade. No caso em que nem todas as variáveis são numéricas, a coluna de tipo diferente será ignorada e a padronização de Gower será aplicada. O quarto argumento é **type** onde é possível passar uma lista de valores especificando os tipos, são eles “ordratio”, transforma a variável de escala de razão para uma ordinal. “logratio” transforma o mesmo tipo para logaritma. “asymm” em binário assimétrico e “sim” simétrico variáveis binárias. O quinto argumento é o **weights** que recebe um vetor numérico opcional com o número de colunas da matriz. Para especificar os pesos para cada variável da matriz, a função também tem valores lógicos que tem a objetivo de avisar quando validações dos tipos das variáveis não são correspondentes.

Valores ausentes são permitidos, pois o algoritmo não inclui essas linhas com informações faltantes. A função divide em dois possíveis casos quando isso acontece, se a métrica é euclidiana ou manhattan, então a dissimilaridade que é retorna é calculada por:

$$\sqrt{\frac{p}{ng}} \quad (18)$$

em que p = número de colunas de x e ng = número de colunas que não possuem nenhuma linha i e j sendo NA. Às vezes com a distância euclidiana encurta o vetor para não produzir valores ausentes, no caso da manhattan o coeficiente é $\frac{p}{ng}$ e em caso de $ng = 0$ a dissimilaridade é NA. Já o segundo caso é para a distância de gower, indicada quando as variáveis são mistas a dissimilaridade entre um par de linhas é a média ponderada das contribuições de cada variável, então temos:

$$d(i, j) = \frac{\sum_{k=1}^p w_k \delta(ij, k) d(ij, k)}{\sum_{k=1}^p w_k \delta(ij, k)} \quad (19)$$

em que w_k são os pesos da k -ésima variável, $\delta(ij, k)$ é uma função indicadora, recebendo 0 ou 1, recebendo 0 se a variável da coluna k está ausente em uma linha i ou j , ou em ambas i e j , no caso da variável binária assimétrica só é atribuído 0 se o par i e j forem 0, caso contrário recebe 1 no caso geral e no caso da variável binária assimétrica. $d(ij, k)$ é a contribuição da variável k para a distância total, isso é a distância entre a observação (i, k) e a (j, k) , se os valores de $d(ij, k)$ for igual quando a variável é nominal ou binária o total para essa dissimilaridade é 0, se não é igual a 1. Porém se $d(ij, k)$ não for nominal ou binária, essa dissimilaridade é a diferença absoluta dos valores, dividida pelo intervalo da variável. Como cada $d(ij, k)$ está entre $[0, 1]$ implica que $d(i, j)$ permanece no intervalo $[0, 1]$, outra observação importante é que se os pesos $w_k \delta(ij, k)$ forem 0 a dissimilaridade é definida como NA.

Em cada uma das seção serão estabelecidos seus objetivos porém foram utilizados objetos criados pela Daisy em 3 situações, nas funções Pam, Agnes e Diana, o objeto gerado pela Daisy pode ser sumarizado.

7.2 PAM

A função Partitioning Around Medoids nomeada como Pam realiza o processo não-hierárquico de agrupamento, ou seja, particiona os dados em k *clusters* em torno dos centróides. Uma primeira observação é que diferente do artigo do J. Macqueen (1967) que se refere ao centro do *cluster* como centróides, na construção da função se referem como "medóides". Então vamos utilizar a expressão que é utilizada pelo livro do Kaufman e rousseeuw (1990), o algoritmo é dividido em duas partes. A primeira é chamada de construção ela segue a seguinte estrutura, primeiro ele considera um objeto i que não foi selecionado e um objeto j , depois deve-se calcular sua dissimilaridade. Então calcule-se a diferença com a dissimilaridade do objeto mais próximo a i , como na expressão (20), com isso se calcula o 'ganho' total somando todos os c_{ji} , depois ele seleciona o objeto j que maximiza a soma de todos os c_{ji} . isso ocorre até o k especificado ser alcançado. Na segunda parte do algoritmo, denominada SWAP, agora tenta-se melhorar o conjunto de observações agrupadas. Isso ocorre pois a segunda etapa simula a troca de elementos para identificar possíveis melhorias. É selecionado alguma observação h , e se é comparado se a distância entre eles, e se c_{jih} é maior, menor ou igual a zero, depois se calcula o c_{jih} que minimize essa soma, se o mínimo dessa soma for negativo, será trocado o par de i e j para i e h , se for positivo ou igual a zero, não se troca e a observação continua no mesmo cluster.

$$c_{ji} = \max(D_j - d(i, j); 0) \quad (20)$$

Na documentação da função é mencionado que essa é uma versão mais robusta do que o algoritmo de k-means. a função tem a seguinte estrutura para ser chamada:

```
pam(x, k, diss = inherits(x, "dist"), metric = c("euclidean", "manhattan"),
medoids = if(is.numeric(nstart)) "random", nstart = if(variant == "faster") 1 else NA,
stand = FALSE, cluster.only = FALSE, do.swap = TRUE,
keep.diss = !diss && !cluster.only && n < 100, keep.data = !diss && !cluster.only,
variant = c("original", "o_1", "o_2", "f_3", "f_4", "f_5", "faster"), pamonce = FALSE,
trace.lev = 0)
```

Assim como a função Daisy, a Pam recebe um x que é uma matriz, porém também é possível receber a matriz de dissimilaridade produzida na Daisy. O segundo argumento é o número de grupos denotados por k . O terceiro argumento é o **diss** que recebe um valor lógico, em caso de TRUE significa que a matriz passada em x é uma matriz de dissimilaridade. O quarto argumento é o **metric** que informa que tipo de distância deverá ser usada se a matriz x não for uma matriz de dissimilaridade, podem ser passadas as distâncias euclidiana ou a manhattan. O quinto argumento é **medoids** pode se passar um vetor inicial com os valores dos medóides, em vez de realizar a primeira parte do algoritmo de construção, ele também pode receber o argumento “random” e aleatorizar essa parte inicial. Se isso acontece, o sexto argumento **nstart** especifica quantos números deverão ser aleatorizados no medoids. O sétimo argumento é o **stand**, que realiza a padronização dos dados caso a matriz x não seja uma matriz de dissimilaridade. O oitavo argumento recebe um lógico, que em caso de TRUE irá calcular somente os *clusters* para as observações, o nome desse argumento é **cluster.only**. O nono argumento é o **do.swap**, que recebe um lógico se deve ou não realizar a fase de troca do algoritmo, **keep.diss** e **keep.data** argumentos para ajustes de alocação de memória. **pamonce** e **variant** são parâmetros que especificam atalhos para o algoritmo pam. Esse atalhos foram algumas melhorias sugeridas por outros pesquisadores, e temos a opção trace.lev que recebe o lógico para saber se é necessário a função retornar todas as etapas de seu processamento.

O objetivo que iremos atribuir a Pam parecerá mais simples, porém ele é um bom caso para se ilustrar de maneira prática a clusterização não-hierárquica. Iremos utilizar dois vetores que apresentaram alguma concentração maior em duas partes distintas do gráfico de dispersão na Figura 4 da seção de análise exploratória foram as variáveis instrumentabilidade e dançabilidade. Tentaremos particionar essa relação, primeiramente estabelecendo o melhor k pelo método da silhueta. Para isso será utilizado a biblioteca factoextra para utilizar a função `fviz_nbclust` que realiza o procedimento. Depois calculando a matriz de dissimilaridade com `Daisy`, para então aplicarmos a Pam e ver sua relação gráfica. Visto a baixa dimensionalidade dos dados. Depois utilizaremos testes estatísticos para determinar se há indícios de diferenças de grupo.

Pam

```
fviz_nbclust(features[,c(2,7)], kmeans, method = "silhouette") # Silhueta
```

```
ds_pam<-daisy(features[,c(2,7)]) # Dissimilaridade
```

```
pm<-pam(ds_pam,2)
```

Criando um data.frame com as variáveis e o vetor com o cluster

```
pmfeatures<-data.frame(features[,c(2,7)],as.factor(pm$clustering))
```

```
names(pmfeatures)<-c("dancabilidade","instrumentabilidade","cluster")
```

Análise gráficas

```
ggplot(pmfeatures,aes(x=instrumentabilidade,y=dancabilidade))+  
  geom_point(aes(col=cluster))
```

Avaliando os grupos

```
m1.mnv<-manova(cbind(instrumentabilidade,dancabilidade)~cluster,data=pmfeatures);
```

```
m1.mnv
```

```
summary(m1.mnv)
```

Figura 10 - Método da Silhueta aplicado a função pam para as variáveis Instrumentabilidade e Dançabilidade

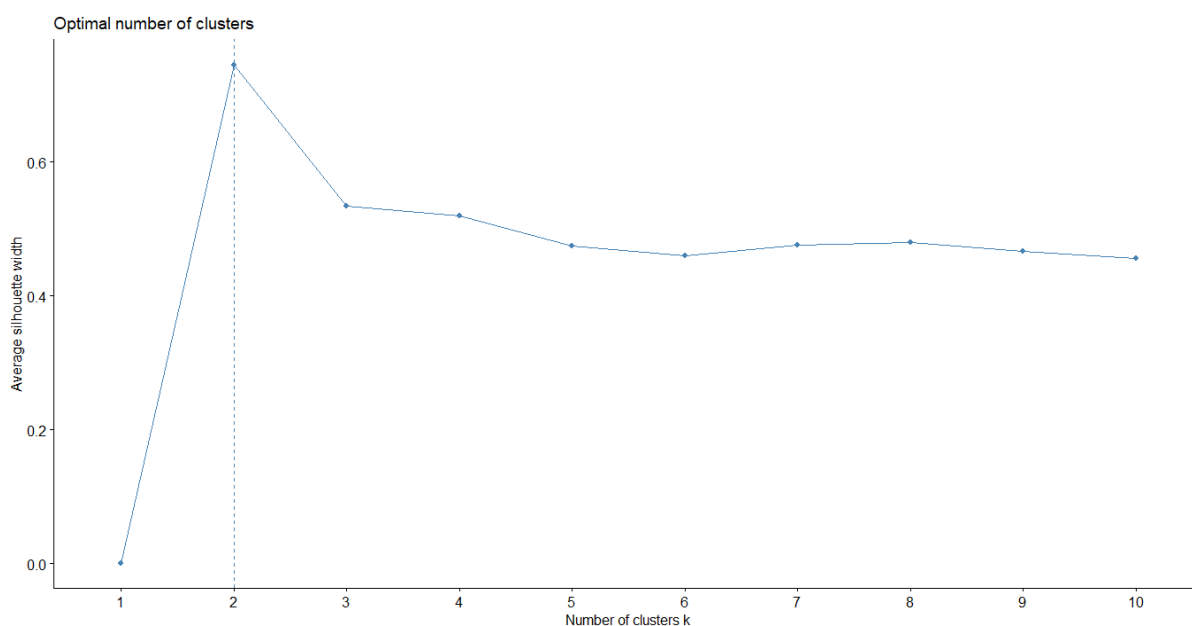
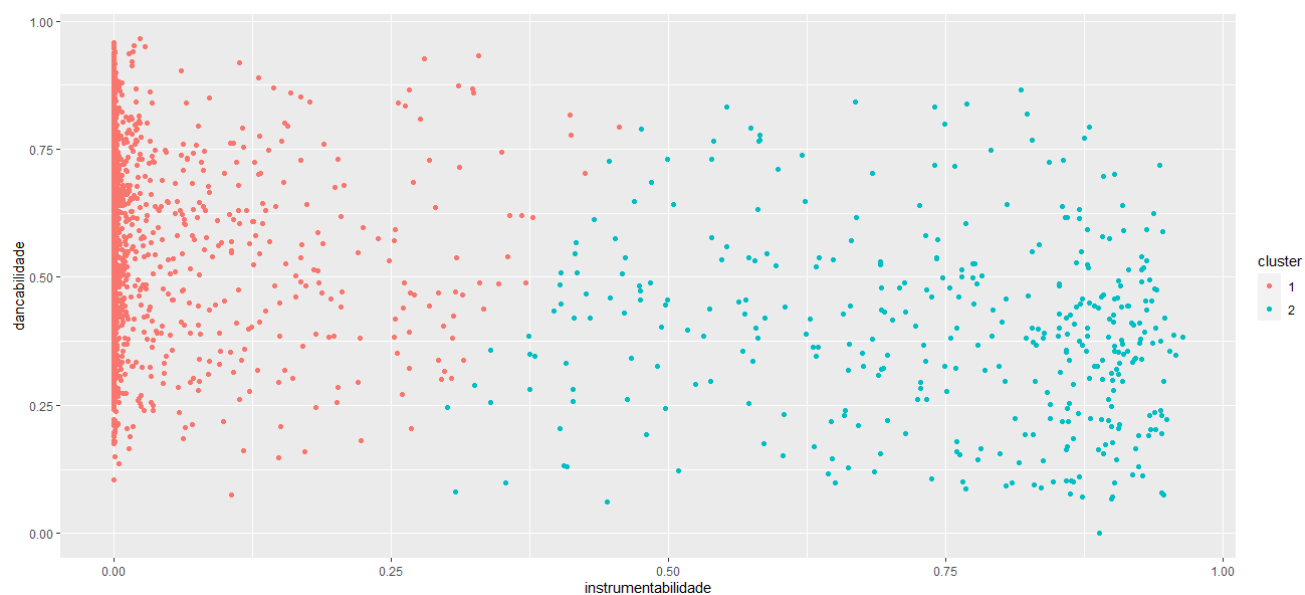


Figura 11 - Gráfico de dispersão 2x2 com a clusterização realizada pela função pam



Quadro 2: Sumarização da Manova para função Pam

	Df	Pllai	Aprox. F	Df	Den DF	Pr(>F)
Cluster	1	0.90059	25503	2	5630	<2.2e-16**
Resíduo	5631					
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1						

Pela Figura 11 temos que número de grupos recomendável é $k=2$ então, foi adotado esse valor é aplicado na função pam. O resultado se mostrou bastante satisfatório visualmente, visto que conseguimos pela Figura 12 uma divisão bem clara em que o grupo 1 tem sua concentração no lado direito e o grupo 2 apresenta uma concentração, porém, no canto esquerdo. Após a clusterização é interessante testar se existem indícios de diferença estatística para os dados. Por isso foi feito a manova com as duas variáveis explicando a variável cluster. Para esse caso em específico em que temos um par de variáveis o teste que é realizado na manova se resume a testa se há diferença estatística entre elas, então ao aplicar obtivemos um p-valor menor que $2.2e-16$. Logo, há indícios para se rejeitar a hipótese de igualdade entre as médias dado o pequeno p-valor em que o nível de significância para α é baixo, algo entre 0 e 0.001. Então os grupos se demonstram diferentes estatisticamente e podemos com isso sugerir um perfil para esses *clusters*. Podemos perceber que a observação com maior instrumentabilidade do *cluster* 1 vai até antes de 0.50 e a observação com o maior valor em relação a dançabilidade do *cluster* dois chega por volta de 0.80. A largura da silhueta média para cada *cluster* ficou sendo 0.76 para o *cluster* 1 e 0.56 para o *cluster* 2 e o *cluster* 1 tem 5226 observações enquanto o *cluster* 2 tem 407 músicas. Então no geral as músicas do banco de dados que foi construído não são tão instrumentais mas porém são dançantes. Isso se dá pelo histograma feito na análise exploratória, uma massa mais distribuída para dançabilidade e uma grande concentração próximo de 0 para a variável instrumentabilidade. E a clusterização captou essa diferença e aplicou a um grupo que preserva valores mais altos para a variável instrumentabilidade e em combinação com dançabilidade apresentou uma menor média e o outro grupo são as demais observações.

7.3 CLARA

O segundo algoritmo de clusterização não-hierárquico se chama Clustering Large Application, apelidado de Clara. Este algoritmo assim como a Pam visa o particionamento dos dados, porém, diferentemente do algoritmo anterior esse modelo tem o objetivo de trabalhar com grandes grupos de dados. Vale lembrar que a Pam tem duas etapas e que os dados passam por um processo, ou seja, seu modelo de trocar observações e comparar suas diferenças pode se tornar muito custoso computacionalmente se tiver uma grande massa de dados, visto que isso é realizado com todas as observações. Então a proposta desse algoritmo é gerar 5 amostras aleatórias, por meio de um gerador pseudo-aleatório que eles buscaram construir de forma independente da máquina que estiver rodando. Kaufman e Rousseeuw (1990) informam que o período desse gerador é de 216. O número de observações variam com o número de *clusters* a contar se estabelece por $40 \text{ observações} + 2 * k$, a primeira amostra é adicionada aleatoriamente e ordenada. Depois de serem formadas é realizada uma comparação entre as observações, a construção das outras amostra leva em consideração os medóides das amostras anteriores. Então é determinado um desenho dos grupos, é utilizado o mesmo algoritmo da pam, agora com k objetos representando os k grupos. Cada observação é atribuída ao *cluster* mais próximo, é o utilizado a distância média como critério. Por último é realizada uma avaliação dos grupos, pegando a distância máxima em relação aos medóides e dividindo pela distância mínima em relação aos medóides. É retornado um valor pequeno indica que o *cluster* está bem agrupado, se for maior que 1 indica que o *cluster* está fraco.

```
clara(x, k, metric = c("euclidean", "manhattan", "jaccard"), stand = FALSE,  
cluster.only = FALSE, samples = 5, sampsize = min(n, 40 + 2 * k), trace = 0,  
medoids.x = TRUE, keep.data = medoids.x, rngR = FALSE, pamLike =  
FALSE, correct.d = TRUE)
```

A função recebe uma matriz x com os dados das variáveis, diferente da Pam não pode-se utilizar a matriz de dissimilaridade criada pela Daisy. O segundo argumento é o k , que é o número de partições que deverá ser realizado. *metric* recebe o tipo de distância que será utilizada, as distâncias possíveis são a euclidiana, manhattan e a jaccard. Outro parâmetro é o *stand* que recebe o valor lógico para padronizar os dados. *cluster.only* recebe o lógico para realizar apenas os valores dos *clusters* ou os objetos. O parâmetro *samples* recebe um inteiro com o valor de números de amostras que devem ser selecionadas para a função. Outro argumento relacionado a amostras é o *sampsize* que estabelece o número de elementos que cada amostra deve ter. Essa função também tem disponível o argumento *trace* que recebe o lógico informando se deve mostrar as etapas do processamento. O argumento *medoids.x* recebe um valor lógico informando se deve retornar os valores dos medóides, em caso de falso argumento seguinte também assume o valor falso que é o *keep.data* que indica se os dados devem ser mantidos no objeto resultante da função. O argumento *rngR* recebe um lógico informando se o gerador aleatório deve ser o do *R* ou o da função clara. O argumento *pamLike* indica se a segunda etapa do algoritmo Pam deve ser realizado, a fase SWAP que ocorre após o desenho realizado pelas amostras. E por último o argumento *correct.d* que recebe um valor lógico referente a uma correção de fórmula originada no código de Fortran.

O objetivo com a função será o mais direto de todas as aplicações, iremos utilizar o data features com os valores numéricos do banco de dados que foi construído e realizar o particionamento dos dados. Utilizando o método da silhueta para determinar o número de partições e depois aplicamos a função clara e avaliaremos seus *clusters* por intermédio de testes.

Clara

Silhueta

```
fviz_nbclust(features, kmeans, method = "silhouette")
```

```
clr<-clara(features,k=4)
```

Clusterização

```

cluster<-as.factor(clr$clustering)

clrfeatures<-data.frame(features,cluster)

# medir a média do cluster para cada atributo

v<-clrfeatures %>%

  group_by(cluster) %>%

    summarise(dancabilidade = mean(dancabilidade),energia = mean(energia),

              volume = mean(volume), cantado = mean(cantado),

              acustica = mean(acustica),

              instrumentabilidade = mean(instrumentabilidade),

              vivacidade = mean(vivacidade), bpm = mean(bpm),

              duracao_ms = mean(duracao_ms)); View(v)

datatable(v); summary(clr)

# avaliação

m2.mnv<-manova(cbind(dancabilidade,energia,volume,cantado,acustica,

                    instrumentabilidade,vivacidade,bpm,duracao_ms,

                    popularidade)~cluster,data=clrfeatures); m2.mnv; summary(m2.mnv)

# Tukey - aplicação de tukey para todas as variáveis

tk1<-TukeyHSD(aov(dancabilidade~cluster,data=clrfeatures));tk1;plot(tk1)

tk2<-TukeyHSD(aov(energia~cluster,data=clrfeatures));tk2;plot(tk2)

```

```
tk10<-TukeyHSD(aov(popularidade~cluster,data=clrfeatures));tk10;plot(tk10)
```

sumarização

```
summary(m2.mnv)
```

Saída para teste da manova:

Quadro 3 - Sumarização da Manova para função Clara

	Df	Pllai	Aprox. F	Df	Den DF	Pr(>F)
<i>cluster</i>	3	1.3986	491.04	30	16866	< 2.2e-16 ***
Resíduo	5629					
Signif. códigos: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1						

Os objetos retornados da função clara são: sample, medoids, i.med, clustering, objective, clusinfo, diss, call, silinfo e data, de acordo com o gráfico da Figura 13 abaixo o número de *cluster* ideal para a clusterização é 4, logo foi aplicado esse valor na função.

Figura 12 - Método da Silhueta aplicado a função clara para as variáveis numéricas do banco de dados

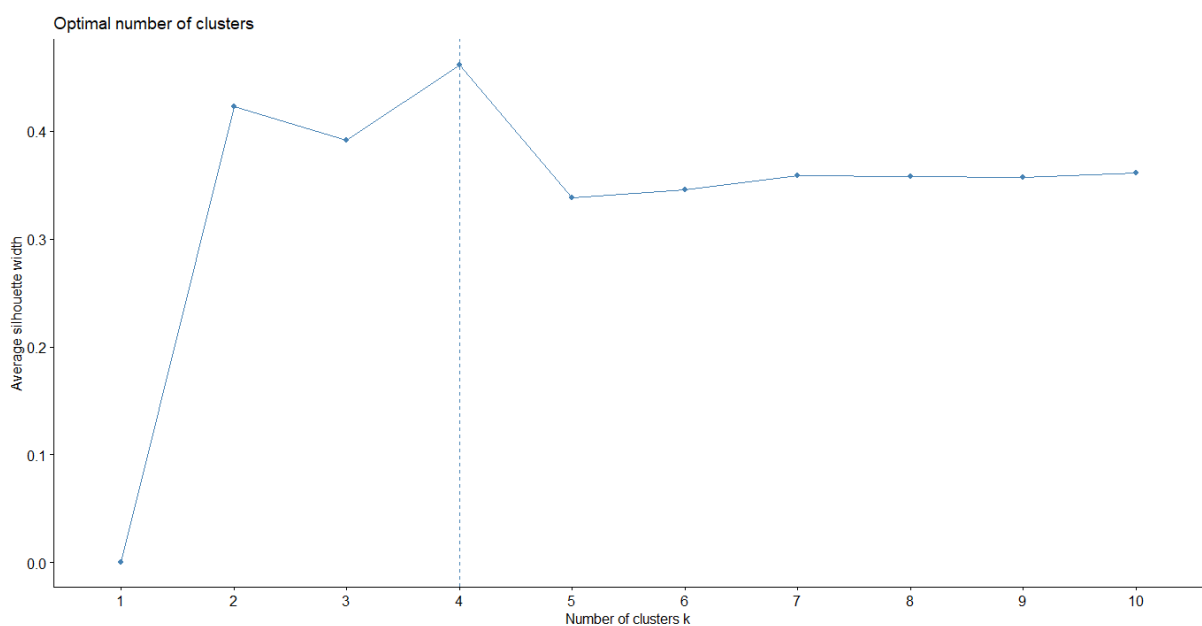
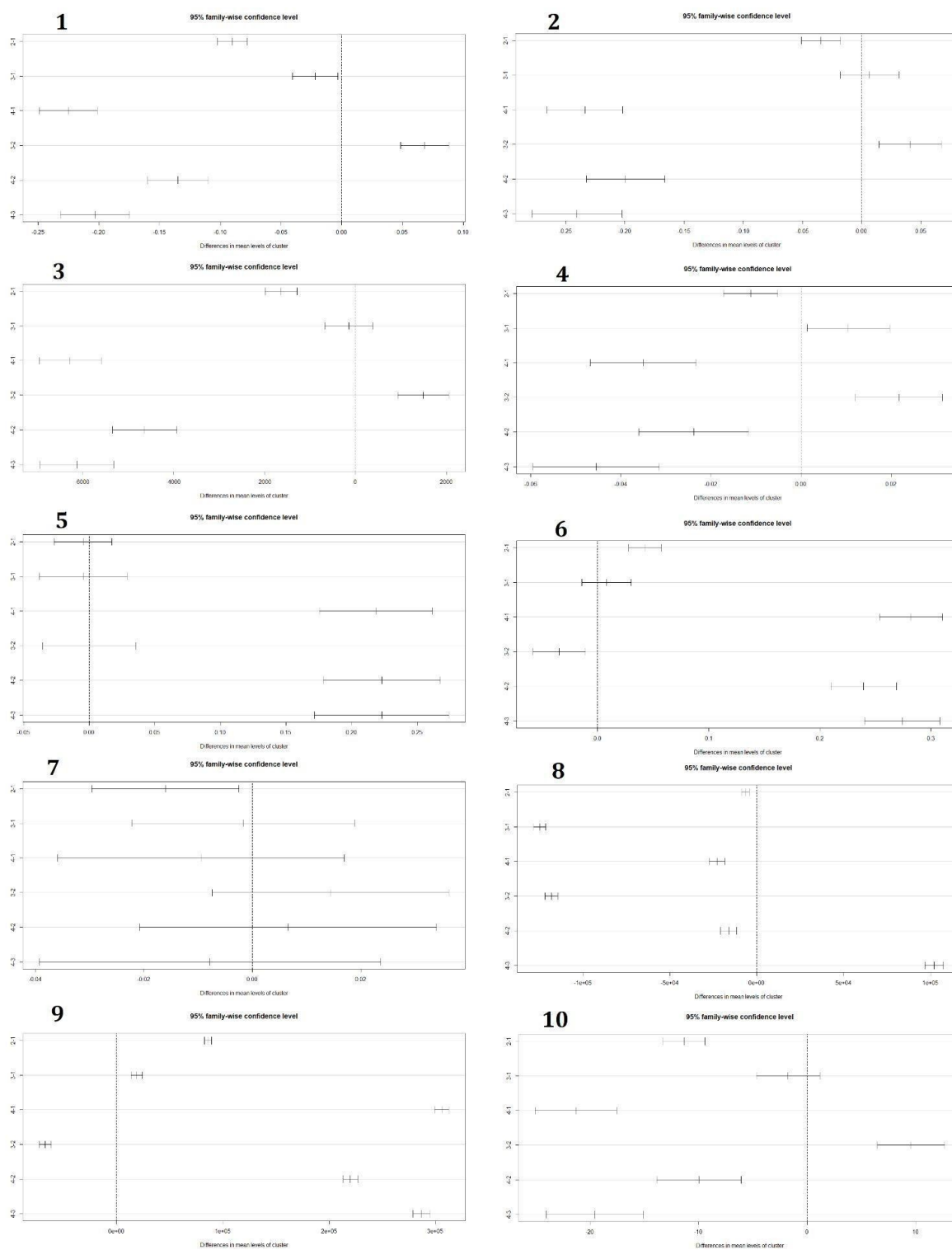


Figura 13 - Intervalos do teste de tukey para cada par de cluster variando para cada variável numérica



1=Dancabilidade; 2=energia; 3=volume; 4=cantado; 5=acustica;
6=Instrumentabilidade; 7=vivacidade; 8=bpm; 9=duracao_ms; 10=popularidade

Quadro 4 - Médias dos *clusters* gerados pela função clara para cada atributo

Cluster	Dançabil.	Energia	Volume	Cantado	Acústica	Instrum.	Vivacidade	Bpm	Duração_ms	Popularidade
1	0.644	0.687	-5709.878	0.084	0.265	0.034	0.207	125641.463	180798.083	66.5213
2	0.554	0.652	-7346.328	0.073	0.260	0.076	0.190	118982.092	266772.694	55.1953
3	0.623	0.693	-5856.085	0.094	0.260	0.042	0.205	736.391	199679.650	64.7701
4	0.419	0.453	-11981.086	0.049	0.483	0.316	0.197	102696.058	486595.757	45.2358

O resultado da aplicação difere da função Pam pois assim como dito no fluxograma, quando temos muitas dimensionalidades gráficos de dispersão não demonstram tão bem o agrupamento. Então na tabela 4, temos as médias para cada variável com isso até pode-se traçar perfis, porém não é uma maneira precisa. Por isso foi realizada a manova para testar as diferenças de médias das variáveis e posteriormente foi feito o teste de tukey para comparação de médias. Seguindo a ideia comentada na seção da manova, acabamos com a mesma situação, em que é realizado a técnica mas ela não é tão informativa em relação a quais médias se diferem e então recorremos a outra.

O primeiro *cluster* obteve 3054 observações, o segundo 1700, o terceiro 561 e por último 318. As suas médias foram 0.493, 0.421, 0.735 e 0.459 respectivamente, então é interessante testar se há diferenças entre esses grupos. Então pela manova temos o $\Pr(>F)$ com o valor de $2.2e-16$ o que nos dá indícios para rejeitarmos a hipótese de igualdade entre as médias. Porém é necessário avaliar qual par que deu diferença, utilizando o teste de tukey podemos averiguar isso pela Figura 14. Temos os intervalos de confiança e uma média central. Em que no eixo y temos os pares de *clusters*, e no gráfico um linha na vertical trastejada centrada no valor zero. Cada intervalo que está ultrapassando essa linha indica igualdade entre as médias. Então temos que nos gráficos 1, 4, 8 e 9 nenhum intervalo obteve o valor em seu intervalo, ou seja, todos os pares se demonstraram significativamente diferentes para os atributos dançabilidade, cantado, bpm e duracao_ms. Nos gráficos 2, 3, 6 e 10 que são das variáveis energia, volume, instrumentabilidade e popularidade, apenas 1 par se mostrou igual e em todos os quatro foram o par 3-1. Os que apresentaram maior igualdade entre os pares foram o 5 e 7 (acústica e vivacidade), 3 pares apresentaram igualdade para a variável

acústica. Para vivacidade apenas um par foi diferente o 2-1, apesar das igualdades, se um par difere, já justificam os *clusters* criados, porém temos indícios que essas variáveis que apresentaram mais igualdade não contribuem tanto para os grupos. É importante também avaliar as correlações das variáveis. Um exemplo é o par 3-1 que foi igual em 6 variáveis, apesar das baixas correlações as variáveis que apresentaram igualdade para esse par são as que apresentaram maior correlação no mapa de calor da Figura 3 criado na seção da análise exploratória. Sendo elas acústica e energia, acústica e volume, instrumentabilidade e energia e instrumentabilidade e volume. Esses 4 pares são as mais correlacionadas, isso consequentemente justifica a utilização da Manova, visto que se todas as correlações fossem baixas não faria sentido a aplicação dessa técnica e sim da ANOVA que se enquadra no caso em que todas as variáveis são independentes.

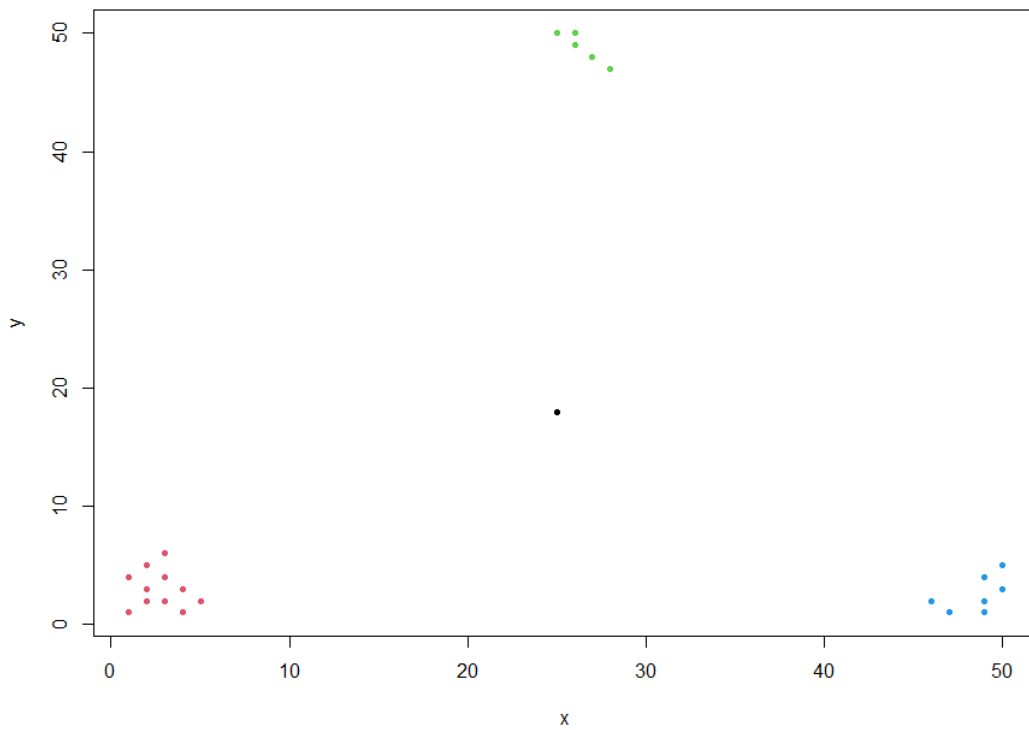
7.4 FANNY

O Algoritmo Fuzzy Analysis nomeado de Fanny é um algoritmo não-hierárquico, que utiliza de maneira difusa para clusterizar. Isso quer dizer que diferente da Pam e da Clara, esse método tem uma certa ambiguidade para os dados no momento de determinar a qual *cluster* a observação irá pertencer. Essa ambiguidade é interessante para pontos de dados que estão em regiões limites de *clusters* ou em regiões que estão isolados dos grupos, pela Figura 15 podemos notar uma possível situação em que isso acontece. Qual seria o *cluster* que deveria ser classificado o ponto preto no centro. Poderíamos formular que essa observação poderia ser um *cluster* único, porém pode haver situações em que temos um número específico de grupos pré-determinados. Então além de ser interessante explorar as características desse ponto, aplicar uma técnica que leve esse tipo de situação em consideração é de suma importância. O método Fuzzy associa a uma observação mais de um *cluster* e cria um grau de “pertencimento” à observação, determinando se uma observação irá ser de um *cluster* A ou de algum *cluster* B. A técnica visa minimizar a função (21). $d(i,j)$ é a dissimilaridade, u_{iu}^2 é o peso que determina o pertencimento da observação, a determinação da

minimização desta função pode ser encontrada a partir da equação de Lagrange, derivando a equação para obtenção do ponto crítico. Isso se dá de maneira recursiva para todo $\mathbf{u}$. Em que $\mathbf{u}$ é maior ou igual a zero e a soma de todos os u' é igual a 1.

$$c = \sum_{u=1}^k \frac{\sum_{i,j=1}^n u_{iu}^2 u_{ju}^2 d(i,j)}{2 \sum_{j=1}^n u_{ju}^2} \quad (21)$$

Figura 14 - Exemplo de pontos em regiões específicas em que algoritmos de clusterização tem dificuldade em agrupar



```
fanny(x, k, diss = inherits(x, "dist"), memb.exp = 2, metric =
c("euclidean", "manhattan", "SqEuclidean"), stand = FALSE,
iniMem.p = NULL, cluster.only = FALSE, keep.diss =
!diss && !cluster.only && n < 100, keep.data = !diss && !cluster.only,
```

$\text{maxit} = 500, \text{tol} = 1\text{e-}15, \text{trace.lev} = 0)$

A função recebe uma matriz x que pode ser uma matriz de dados ou dissimilaridade calculada pela função Daisy. existe uma certa tolerância para dados faltantes em matrizes de dados e em caso de matriz de dissimilaridade não há. O segundo argumento é o k que será o número de grupos que os dados serão particionados. O **diss** informa se o que foi passado é uma matriz de dados ou uma matriz de dissimilaridade, se receber TRUE é ele utiliza a configuração para dissimilaridade e ignora outros parâmetros. Como a **metric**, parâmetro esse que se recebe uma matriz de dados irá utilizar para calcular as distâncias, ela recebe as distâncias euclidiana, manhattan e SqEuclidean. **memb.exp** recebe a especificação do expoente de associação usado no critério de ajuste das observações, se o parâmetro é $r = 1$ o grupo será nítido, e apresenta uma convergência lenta, no caso de $r = 2$, aumenta-se o grau de imprecisão, e dependendo desse grau um aviso é emitido pela função para se escolher um r menor. A função **stand** padroniza os dados, porém se for uma matriz de dissimilaridade isso é ignorado. O argumento **ini.Men.p**, é uma matriz nula ou numérica que é utilizada para adesão inicial, a matriz é não negativa e suas linhas somam 1. **cluster.only** recebe um valor lógico, se for TRUE nenhuma informação da silhueta é calculada, ela também tem parâmetros para alocação de memória. **keep.diss** e **keep.data** e temos também **maxit** e **tol** que são número máximo de iterações e tolerância padrão, sendo por base $\text{maxit} = 500$ e $\text{tol} = 1\text{e-}15$, e temos a **trace.lev** que informa as etapas de processamento do algoritmo.

O objetivo da aplicação para Fanny será associado aos tons das músicas do banco de dados, em teoria musical os tipos de campos harmônicos fornecem características para determinadas escalas, porém isso se torna relativo com base na tônica da música. Então a ideia é explorar que tipo de agrupamento será formado, além disso, utilizaremos as médias dos dados do banco features e o vetor de tons das músicas.

Para essa aplicação foi construída a matriz de dissimilaridade dentro da função fanny, pois apresentou uma melhor performance em tempo e qualidade de resultados. A construção do banco de dados foi feita com base na média das músicas de cada nota. Ao todo temos 12 tons, e cada um apresenta uma média para cada atributo selecionado, com isso será aplicada a função fanny com $k = 3$, realizado de maneira recursiva apresentou o melhor agrupamento, a

métrica usada foi SqEuclidean, ou seja, o quadrado da distância euclidiana. A fanny retorna um objeto com os itens, membership, coeff, memb.exp, clustering, k.crisp, objective, convergence, diss, call, silinfo e data, abaixo temos o código da formação do banco de dados e os resultados retornados:

Nome das *playlist*

```
playlist_nomes<-names(table(data_ws$tom))  
  
features2<-data.frame(features,data_ws$tom)
```

Calculando as médias de cada *playlist* para todos atributos desejados

```
medias_tons <- features2 %>%  
  
  group_by(data_ws.tom) %>%  
  
  summarise(n=n(),  
  
    dancabilidade = mean(dancabilidade), energia = mean(energia),  
  
    volume = mean(volume), cantado = mean(cantado),  
  
    acústica = mean(acustica), instrumentabilidade = mean(instrumentabilidade),  
  
    vivacidade = mean(vivacidade), bpm = mean(bpm),  
  
    duracao_ms = mean(duracao_ms))
```

Organização dos dados

```
medias_tons<-medias_tons[,-c(1,2)]  
  
medias_tons<-data.frame(medias_tons,row.names = playlist_nomes)
```

Fanny

```
fnn<-fanny(medias_tons,3,diss=F,metric = "SqEuclidean")
```

summary(fnn); plot(fnn)

a saída da summary(fnn) se encontra no apêndice A e plot(fnn) se encontram abaixo:

Figura 15 - Clusterização dos tons musicais do banco de dados com as músicas do *spotify* com a função Fanny

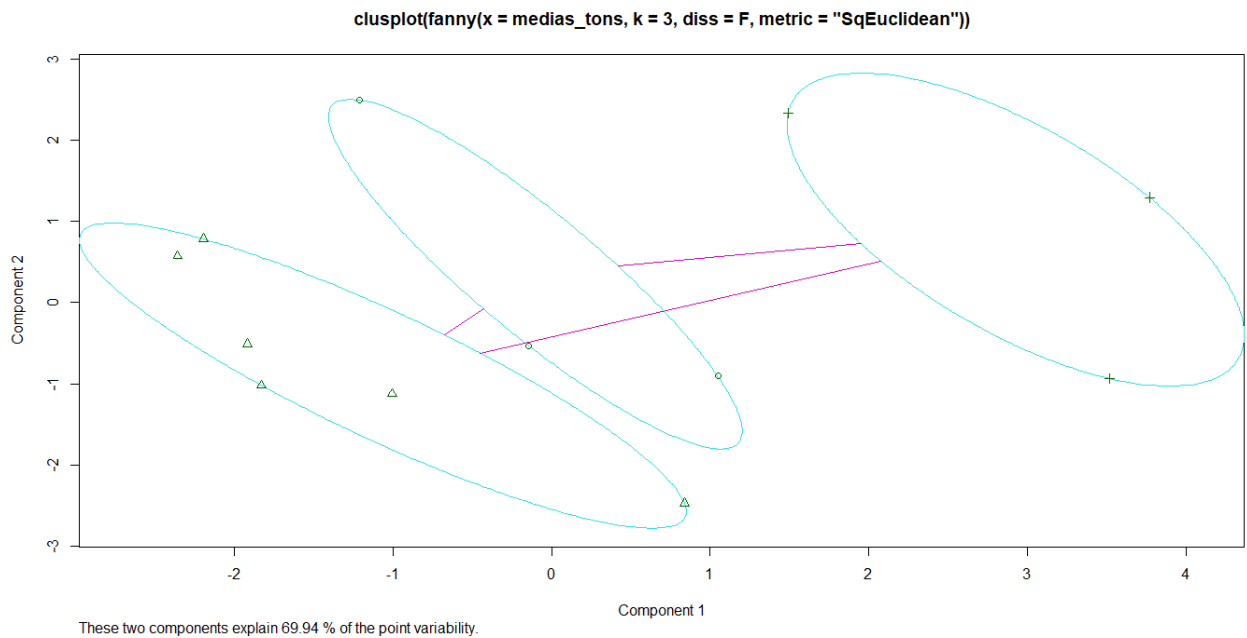
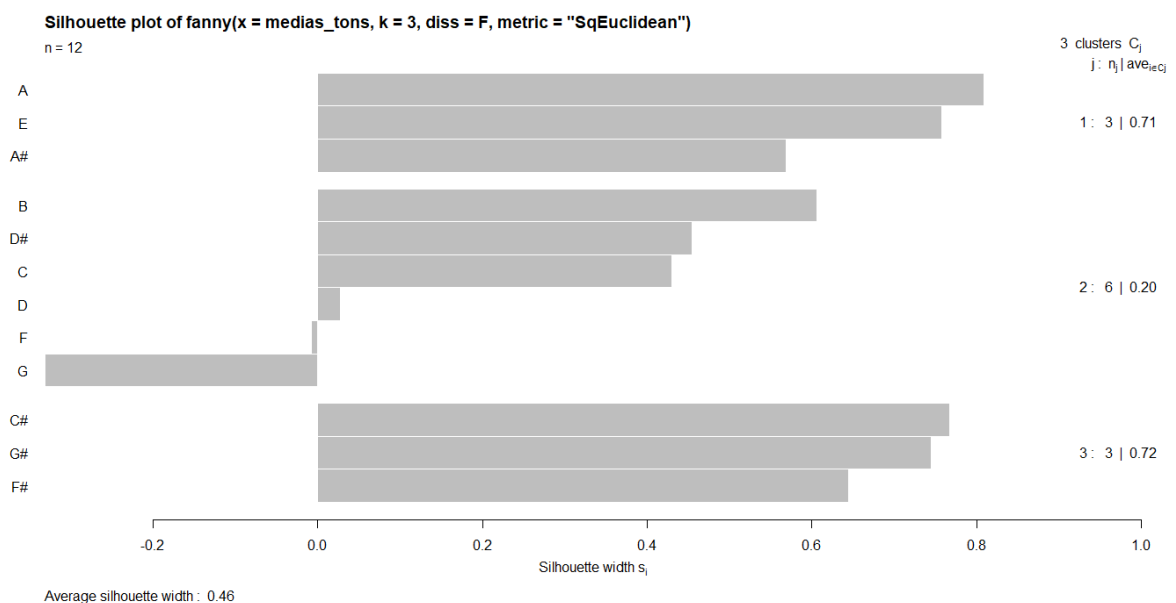


Figura 16 - Gráfico da Silhueta dos *clusters* gerados pela função Fanny



O objeto `Summary(fnn)` retorna primeiro algumas informações sobre o algoritmo. Como o número de observações (12) a tolerância e o número de iterações máximas que foram usadas as padrões, em seguida temos a `membership coefficients (in %, rounded)`, que retorna a porcentagem de cada observação obteve para cada *clusters*. E a maior porcentagem faz com que a observação pertença ao cluster. A observação que mais se aproximou foi a observação Dó (C) em que obtivemos 25, 41 e 34, as outras ficaram bem explícitas e houve pouca dificuldade na decisão. Em seguida temos para cada observação qual foi o seu *cluster* de destino. Depois a silhueta com suas respectivas larguras, obtivemos as médias para os *clusters* e uma média geral, temos também uma sumarização da matriz de dissimilaridade.

O primeiro gráfico, plotado na Figura 16, nos fornece um *clusplot* da *fanny*. É interessante que o *clusplot* gerou duas componentes que tentam explicar a variabilidade dos dados. Isso com esses componentes explicam 69.94% dos dados. A clusterização ficou bastante interessante, visto que temos 3 grupos independentes, que não se interseccionam. O pacote *cluster* fornece a estrutura que criam as elipses que englobam as observações dentro dos *clusters*. No segundo gráfico na Figura 17, temos um gráfico da silhueta e as largura das observações. No canto direito temos o *cluster* com o número de observações e suas médias, o *cluster* 1 tem 3 observações com média 0.71. O segundo *cluster* tem 6 observações com média de 0.2 e as notas de Fá e Sol obtiveram larguras para as silhuetas negativas. E o último *cluster* obteve 3 observações com média de 0.32. Em geral a clusterização construiu bons grupos com características particulares, cabe investigar um possível estudo futuro sobre a relação desses tons. Vale ressaltar que ao tentar utilizar $k = 6$ o algoritmo não apresenta um erro, e o particionamento com 3 grupos apresentou a melhor divisão e uma boa utilização para o algoritmo *fanny*, visto que existem pontos em regiões críticas.

7.5 AGNES

O algoritmo Agglomerative Nesting apelidado de Agnes é o primeiro método Hierárquico do pacote e do livro do Kaufman e Rousseeuw (1990), porém ele é o único que

realiza o processo de forma aglomerativa, ou seja, o algoritmo une os *clusters* ao decorrer das suas etapas de cálculo. Inicialmente cada observação - linha do banco de dados - são consideradas como um cluster, a partir da segunda etapa cada *cluster* irá se unir com outro *cluster* dado um critério de dissimilaridade. Esse critério pode ser a média, o máximo, o mínimo e entre outros critérios, no trabalho do Kaufman e Rousseeuw (1990) mencionado no início deste parágrafo. É mencionado que a média é a definição padrão para essa aglomeração, para aplicar esses critérios é utilizado o resultado do grupo sokal e michener que é chamada de UPGMA que em tradução livre é o método da média por pares de grupos não ponderados. A expressão (22) demonstra esse método, em que R e Q são *clusters* e |Q| e |R| São os números de elementos nos *clusters*, e d(i,j) é a dissimilaridade das observações, então dado esse método serão calculadas a cada etapa as combinações dos *clusters* atuais da etapa e será utilizado um critério para determinar como será o agrupamento.

$$d(Q,R)=\frac{1}{|Q||R|} \sum_{i \in R \text{ e } j \in Q} d(i,j) \quad (22)$$

```
agnes(x, diss = inherits(x, "dist"), metric = "euclidean", stand = FALSE, method = "average", par.method, keep.diss = n < 100, keep.data = !diss, trace.lev = 0)
```

A função Agnes recebe os parâmetros *x* uma matriz de dissimilaridade ou algum banco de dados, se o argumento for um banco de dados é permitido dados faltantes, para o caso de uma matriz de dissimilaridade não é permitido observações faltantes. O segundo argumento é o **diss** que recebe um valor lógico TRUE e FALSE, caso receba o valor TRUE, a matriz é considerada de dissimilaridade, em caso de FALSE, é considerado um banco de dados. O terceiro é **metric** que recebe o tipo de distância, sendo as disponíveis a Euclidiana e a manhattan, se o argumento x passado e o argumento diss receberem uma matriz de dissimilaridade esse argumento não é utilizado. O quarto argumento é o **stand** que recebe também um valor lógico, se receber verdadeiro ele irá padronizar os dados. O quinto argumento é o **method** que recebe que tipo de critérios a função irá utilizar para realizar a

divisão, por padrão o método "average" calcula a média das dissimilaridades entre os pontos de dois *clusters*. Temos também os seguintes argumentos: o método "single" é utilizado a menor dissimilaridade entre um ponto de um *cluster* e outro cluster. O método "complete" que utiliza a maior dissimilaridade entre um ponto de um *cluster* e ponto de um segundo *cluster* utilizando o critério de forma contrária ao método "single". Outro método que se torna mais complexo é o "flexible" esse método utiliza o sexto argumento *par.method* como ajuste para esse argumento, "flexible" utiliza a fórmula de lance williams (23)

$$D(C_1, C_2, Q) = \alpha_1 D(C_1, Q) + \alpha_2 D(C_2, Q) + \beta D(C_1, C_2) + \gamma |D(C_1, Q) - D(C_2, Q)| \quad (23)$$

em que $\alpha_1, \alpha_2, \beta$ e γ são parâmetros passados pelo argumento *par.method* em forma de vetor, é comentado também que dependendo de como for passado esses parâmetros pode-se formar outros métodos. Há também o método "gaverage" que serve com uma generalização do método "average". Os argumentos *keep.diss* e *keep.data* podem melhorar a alocação de memória para o processamento da função e o argumento *trace.lev* informa se deverá mostrar as etapas de processamento do algoritmo.

Então para o objetivo da aplicação desse algoritmo, queremos expressar a dissimilaridade dos artistas mais recorrentes no banco de dados e variar o tipo de critério para a aglomeração dos *clusters*. Então será montado o banco de dados com os artistas que apareceram com pelo menos 10 músicas no banco de dados, foi calculada a média de cada atributo para cada artista e foi aplicado na função Agnes. A primeira aplicação dela será a que apresentou os melhores *clusters*, e as outras serão as variações.

Agnes - Artista - Separando os artistas com mais de 10 músicas

```
fastdata <- as.data.frame(table(data_cat$artista))
```

```
fastdata <- fastdata[fastdata$Freq>10, ]
```

```
fastdata <- fastdata[fastdata$Freq<100, ]
```

```
artist<-fastdata$Var1
```

```
m<-NULL
```

```
for(i in 1:length(data_ws$n0)){
```

```
  for(j in 1:length(artist)){
```

```
    if(data_ws$artista[i]==artist[j]){m[i]<-data_ws$n0[i] }
```

```
  }}; m<-na.omit(m); m; features2<-data_ws[m, ]
```

```
#Calculando a média de cada artista
```

```
artistas_features<-features2 %>%
```

```
  group_by(artista) %>%
```

```
  summarise(dancabilidade = mean(dancabilidade),energia = mean(energia),
```

```
  volume = mean(volume),cantado = mean(cantado), acustica = mean(acustica),
```

```
  instrumentabilidade = mean(instrumentabilidade),
```

```
  vivacidade = mean(vivacidade), bpm = mean(bpm),
```

```
  duracao_ms = mean(duracao_ms))
```

```
# construindo o banco de dados
```

```
artist2<-artistas_features$artista; artistas_features<-artistas_features[, -1]
```

```
artistas_features<-data.frame(artistas_features,row.names = artist2);rm(artist2)
```

```
View(artistas_features)
```

```
# aplicação da Agnes - aplicando a Dayse e a Agnes
```

```
ds_agnes <- daisy(artistas_features,stand = T)
```

```
gns<-agnes(ds_agnes,diss=T);plot(gns)
```

```
# variando critério - single
```

```
gns<-agnes(artistas_features,diss=F,stand = T,metric = "euclidean",method = "single");  
plot(gns)
```

```
# variando critério - complete
```

```
gns<-agnes(artistas_features,diss=F,stand = T,metric = "euclidean", method = "complete");  
plot(gns)
```

```
# variando critério - flexible
```

```
gns<-agnes(artistas_features,diss=F,stand = T,metric = "euclidean", method = "flexible",  
par.method = c(0.9,0.9,0.9,0.9)); plot(gns)
```

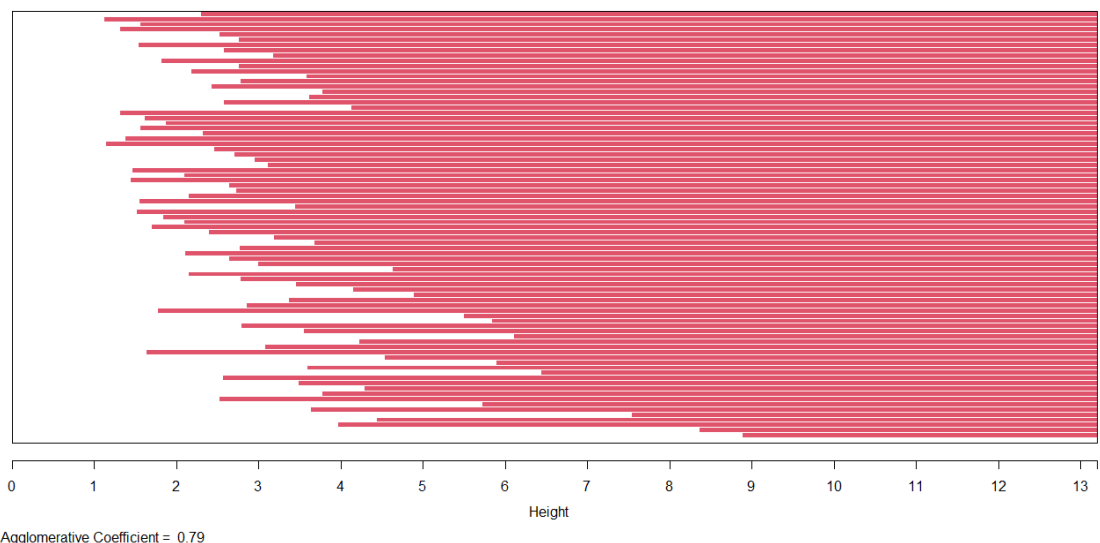
```
# variando critério - gaverage
```

```
gns<-agnes(artistas_features,diss=F,stand = T,metric = "euclidean", method = "gaverage");  
plot(gns)
```

Agora iremos ver as saídas desses códigos, porém será omitido o primeiro gráfico das variações de critérios. a Agnes retorna um objeto com os elementos order, height, ac, merge, diss, call, method, order.lab e data, ao plotar temos dois gráficos.

Figura 17 - Gráfico tipo Banner da função Agnes para o conjunto de dados

Banner of agnes(x = ds_agnes, diss = T)



Dendrogram of agnes(x = ds_agnes, diss = T)

Height

ds_agnes
Agglomerative Coefficient = 0.79

AC/DC
Aerosmith
Bruce Springsteen
Nirvana
U2
Coldplay
Jimi Hendrix
Van Halen
The Rolling Stones
Santitas
Luan Platin
Green Day
Harry Styles
Red Hot Chili Peppers
Lenny Kravitz
The Beatles
Ozzy Osbourne
P.O.D.
Pearl Jam
The Clash
Dua Lipa
Ava Max
The Weeknd
Ed Sheeran
Gustavo Lima
Måneskin
PEDRO SAMPÃO
J Balvin
Ariana Grande
Luisa Sonza
Cardi B
Post Malone
Bad Bunny
Bruno Mars
Shawn Mendes
The Lumineers
Israel & Rodolfo
Zé Vaqueiro
João Gomes
Natti Natasha
Vitor Fernandes
Rai Saia Rodada
Wesley Salgado
Credence Clearwater Revival
Marília Mendonça
Maroon 5
MC Zaqueu
Jorge & Mateus
Bruno e Marrone
Vitor e Luísa
Os Barões da Pisadinha
Linkin Park
Marques & Kauán
Diego & Victor Hugo
Lewis Capaldi
Olivia Rodrigo
Taylor Swift
Fanny Lu
Sallia Pearch
Fernando & Sorocaba
Hugo & Guilherme
MC Kevin o Chris
Pisa Star
Calvin Harris
Maestric
Riton
Justin Bieber
Lea Zermani
Maléfica
David Bowie
The Who
Queen
Desafinada
Guns N' Roses
Metallica
Don Henley
Mc Pato
Mc Cherry
Mc Poze do Rodo
Pop Smoke
Thiaguinho
Pineapple Storm
Billie Eilish

Dendrogram of agnes(x = artistas_features, diss = F, metric = "euclidean", stand = T, method = "single")

Height

1 2 3 4 5 6 7 8

Artists (from left to right):

- Aerosmith
- Bruce Springsteen
- Nirvana
- U2
- Green Day
- AC/DC
- Alok
- The Clash
- Ed Sheeran
- Gustavo Lima
- Manskin
- The Weeknd
- Ariana Grande
- Dua Lipa
- Post Malone
- Shawn Mendes
- The Kid Laroi
- PEDRO SAMPAYO
- Bad Bunny
- Galantis
- Luan Santana
- Israel & Rócoli
- Vítor Fernandes
- Rai Sala Rotada
- NATTIAN
- Zé Neto & Cristiano
- Harry Styles
- Mathaus & Kauan
- Jimmi Helder
- Van Halen
- MC Zauzin
- Marília Mendonça
- Red Hot Chili Peppers
- Jorge & Mateus
- Os Barões Da Píladina
- Credence Clearwater Revue
- The Roots
- The Beatles
- The Who
- Bruno & Marrone
- Verônica
- David Bowie
- Guns N' Roses
- Lewis Capaldi
- Olivia Rodrigo
- Taylor Swift
- Wesley Safadão
- Kenrick Chan
- Ozzy Osbourne
- Hugo & Guilherme
- Fernando & Sorocaba
- Diego & Victor Hugo
- Linkin Park
- Heart
- Calvin Harris
- Madison Beer
- Justin Bieber
- MC Kevin o Chris
- Dr. Smoke
- Ed Sheeran
- Deep Purple
- Dor Hentley
- Grisa Star
- Leão Zecchin
- Madley Crie
- Mc Poze do Rodo
- mc Jhenny
- Almaguinho
- Pineapple StormTV
- Billie Eilish

artistas_features

Agglomerative Coefficient = 0.71

Dendrogram of agnes(x = artistas_features, diss = F, metric = "euclidean", stand = T, method = "complete")

Height

0 5 10 15

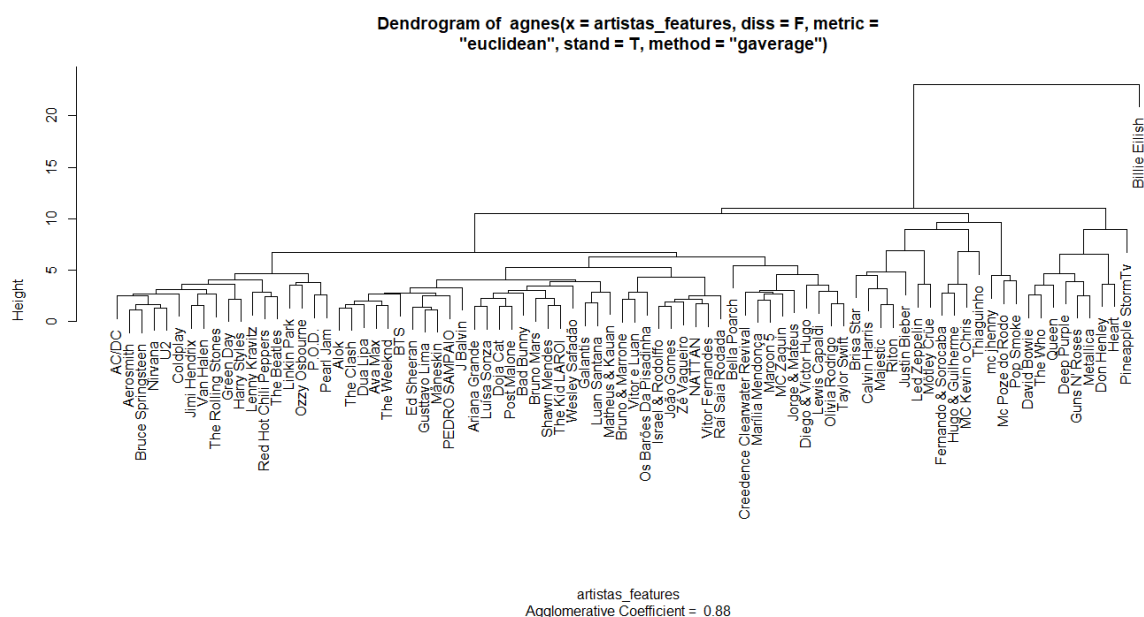
artistas_features
Agglomerative Coefficient = 0.82

Dendrogram of agnes(x = artistas_features, diss = F, metric = "euclidean", stand = T, method = "flexible", par.method = c(0.9, 0.9, 0.9, 0.9))

Height

artistas_features
Agglomerative Coefficient = 1

Figura 22 - Gráfico tipo Dendrograma para a função Agnes aplicado aos artistas com mais de dez músicas no conjunto de dados com método gaverage



Na Figura 18 temos um gráfico que será melhor trabalhado na seção da Diana. Porém ele é o complementar do gráfico da Figura 24, o começo da linha vermelha representa o momento da união do cluster, porém ele não é tão intuitivo quanto o Dendrograma da Figura 19. No lado esquerdo temos um *cluster* partindo da banda AC/DC até Pearl Jam, temos bandas de Rock e suas variantes que surgiram nas últimas décadas do século XX, no centro esquerdo temos um *cluster* maior que começa com artista Alok e vai até a dupla Jorge e Mateus. Contemplando o primeiro grupo de artistas voltados a músicas comerciais. A partir dos próximos *cluster* começamos a ter artistas com características bem diferentes próximos uns dos outros. Temos por exemplo a banda Linkin Park próximo a banda Barões da Pisadinhas, grupos totalmente diferentes mas que ainda sim são um produto comercial que aparentemente preservam algum grau de dissimilaridade em suas médias maior que outros artistas que normalmente imaginamos uma semelhança maior. Então de Bruno e Marrone até pineapple smoke tv temos um grupo bem confuso e por último separado de todos temos a artista Billie Eilish. Sobre as outras variações. O tipo single (20) que utiliza a menor dissimilaridade médias entre os *clusters*, apresenta uma aglomeração que vai crescendo

gradativamente, apesar de ter algumas aglomerações mais distantes. Já no caso do critério complete (21), que utilizar as maior dissimilaridade média entre os *clusters* temos uma formação de *clusters* mais próximo da padrão average. No caso flexible (22) foi forçado os parâmetros para os extremos então a temos *cluster* muito bem divididos porém com baixa hierarquia. No último caso, que utiliza um método generalizado das médias (23), temos um resultado parecido com o método padrão com alterações nos grupos de artistas. Esse se demonstrou o melhor a ser comparado com o método padrão.

7.6 DIANA

O Programa Divisive Analysis Clustering, nomeado como Diana, tem o objetivo formar agrupamentos hierárquicos com base em divisões dos dados. Isso ocorre de maneira recursiva, utilizando a matriz de dissimilaridade. No primeiro passo todas as observações estão no mesmo grupo, no segundo passo é calculado a dissimilaridade média para cada variável. A que obtiver a maior média é separada e o processo continua com as demais variáveis, realizando as divisões. Esse método é descrito pelo Kaufman e Rousseeuw (1990) na seção relacionada a Diana como método iterativo de Macnaughton-Smith, Ele se constrói com a ideia de um conjunto A com i elementos e um conjunto B vazio. A cada etapa seleciona-se o elemento i que maximiza a função (24). O valor i que maximiza a (25) for estritamente positivo, então o processo continua, se for negativo ou zero, se para o processo.

$$d(i, A \setminus \{i\}) = \frac{1}{|A| - 1} \sum_{j \in A, j \neq i} d(i, j) \quad (24)$$

$$d(i, A \setminus \{i\}) - d(i, B) = \frac{1}{|A| - 1} \sum_{j \in A, j \neq i} d(i, j) - \frac{1}{|B|} \sum_{h \in B} d(i, h) \quad (25)$$

```
diana(x, diss = inherits(x, "dist"), metric = "euclidean", stand = FALSE,
stop.at.k = FALSE, keep.diss = n < 100, keep.data = !diss, trace.lev = 0)
```

A função *Diana* recebe como parâmetros $\mathbf{x}$ uma matriz de dissimilaridade ou um banco de dados, para o caso de um banco de dados é permitido dados faltantes, para o caso de uma matriz de dissimilaridade não é permitido observações faltantes. O segundo argumento é o *diss* que recebe um valor lógico, em caso de o valor TRUE, a matriz é considerada de dissimilaridade, em caso de FALSE, é considerado um banco de dados. O terceiro é *metric* que recebe o tipo de distância, sendo as disponíveis a Euclidiana e a manhattan. Se o argumento $\mathbf{x}$ passado e o argumento *diss* receberem uma matriz de dissimilaridade esse argumento não é utilizado. O quarto argumento é o *stand* que recebe também um valor lógico, se receber verdadeiro ele irá padronizar os dados, porém apenas se for uma matriz de dados, para matriz de dissimilaridade é ignorado, existem outros três argumentos. Porém um deles não está implementado e os outros são sobre alocação em memória caso necessário. E o último *trace.lev* indica um diagnóstico durante o algoritmo.

Para a aplicação da função percebemos que o foco é em dividir com base nos cálculos das dissimilaridades para estabelecer suas hierarquias, então será proposto um objetivo utilizando o banco de dados formado neste trabalho. Podemos averiguar que tipo de *playlist* tem as dissimilaridades mais discrepantes umas das outras. Para isso iremos fazer uma manipulação no nosso data frame, iremos calcular cada média das variáveis numéricas para cada *playlist*. as variáveis são: dançabilidade, energia, volume, cantado, acústica, instrumentabilidade, vivacidade, bpm e duracao_ms. Posteriormente será organizado o banco de dados, calculada a matriz de dissimilaridade com a função *Daisy*. Para esse banco de dados de médias da *playlist* a função *Daisy* foi utilizada padronizando os dados e utilizando a distância euclidiana. Após essa etapa, foi passada para a função *Diana* com a matriz de dissimilaridade e o argumento *diss* =TRUE. O código e as saídas de interesses se encontram abaixo:

```
# Nome das playlist
```

```
playlist_nomes<-names(table(data_ws$playlist))
```

```
# Calculando as médias de cada playlist para todos atributos desejados
```

```
medias_play<-data_ws %>%
```

```

group_by(playlist) %>% summarise(n=n(),dancabilidade = mean(dancabilidade),
                                energia = mean(energia), volume = mean(volume),
                                cantado = mean(cantado), acustica = mean(acustica),
                                instrumentabilidade = mean(instrumentabilidade),
                                vivacidade = mean(vivacidade), bpm = mean(bpm),
                                duracao_ms = mean(duracao_ms))

# organização dos dados
medias_play<-medias_play[,-c(1,2)]
medias_play<-data.frame(medias_play,row.names = playlist_nomes)

# Matriz de dissimilaridade com variáveis mistas
ds_diana <- daisy(medias_play,stand = T)

# aplicação da Diana
dn <- diana(ds_diana,diss=T)
plot(dn)

```

A saída da função Diana, retorna vários objetos, sendo eles order, height, dc, merge (um banco de dados), diss, call, e order.lab. Ao aplicar o algoritmo no console do programa R, então ele imprime o gráfico da Figura 24, após o print do gráfico ele solicita novamente que seja apertada a tecla, retornando o gráfico da Figura 25.

Figura 23 - Gráfico tipo Banner da função Diana para o conjunto de dados

Banner of diana(x = ds_diana, diss = T)

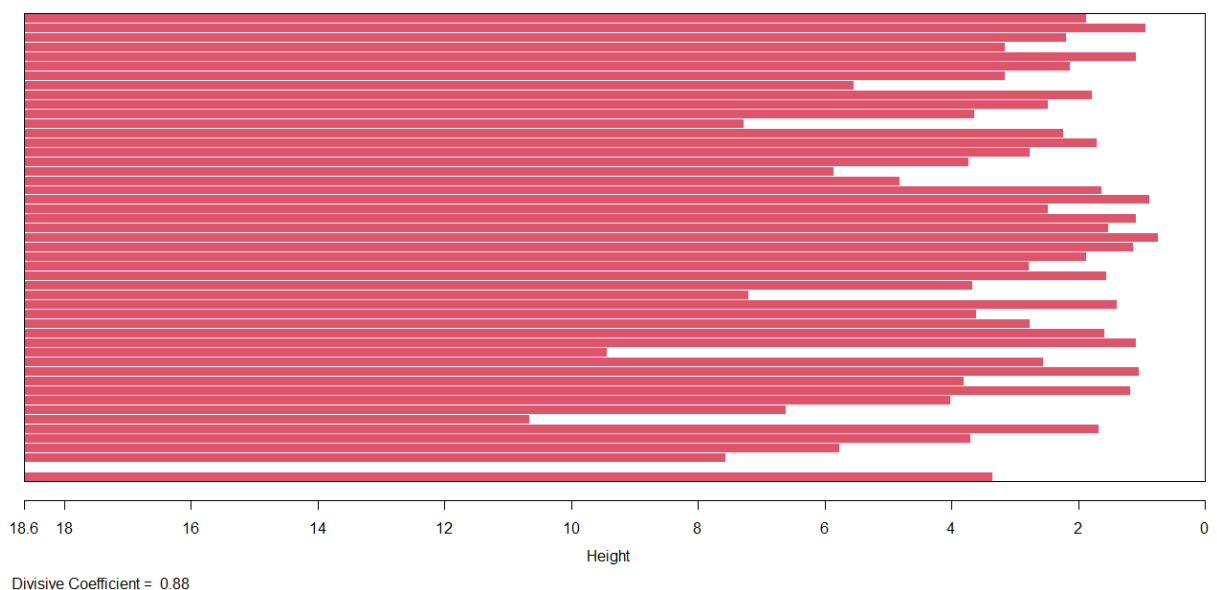
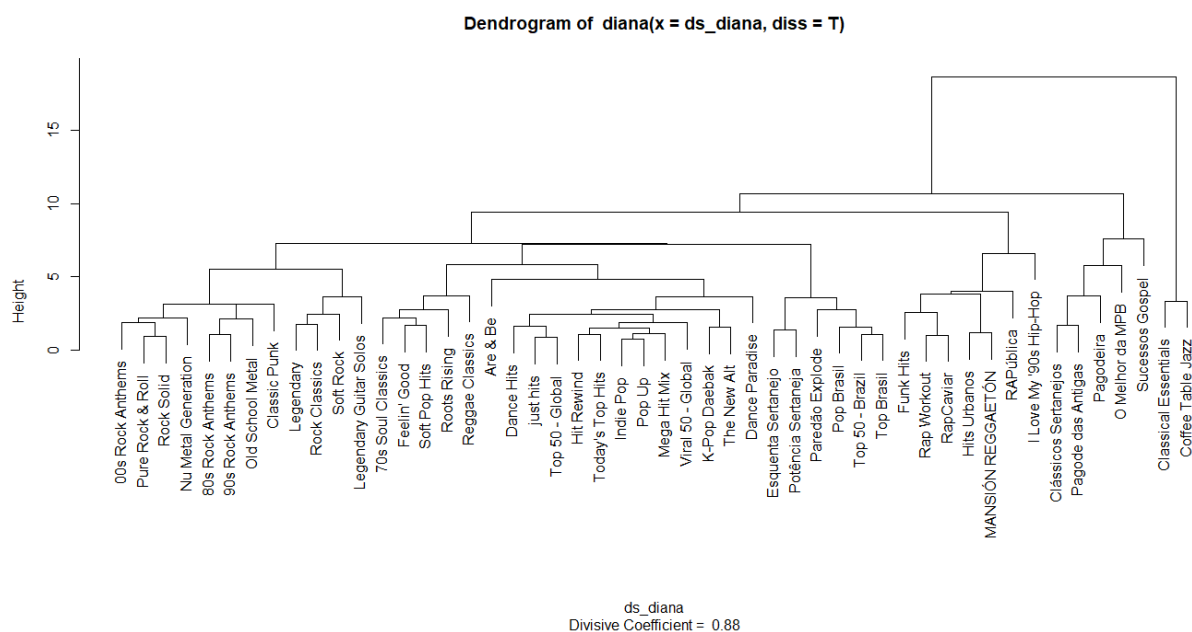


Figura 24 - Gráfico tipo Dendrograma para a função Diana aplicado às médias das *playlists* para cada atributo



O gráfico da Figura 24 expressa as alturas de cada uma das divisões. Uma Figura foi produzida e consta no apêndice C, ilustrando o que ocorre. Em relação ao gráfico da Figura 25 produzido pela Diana, temos um dendrograma com as *playlists*. partindo com a análise do eixo height (altura) maior indo para os menores, podemos perceber que a primeira divisão que é feita é entre as *playlists* Classical Essentials e Coffee Table Jazz em um segmento e todas as outras *playlists* do banco de dados. Essas duas apresentaram maior dissimilaridade na primeira etapa. No segundo segmento em que todas as *playlists* estão, a próxima divisão separa as seguintes playlists das outras. Sucessos gospel, O melhor da MPB, Pagodeira, Pagodes das Antigas e Clássicos Sertanejos, esses dois primeiros passos separam *playlists* que preservam características bem particulares uma das outras. O Jazz, a música clássica, o Pagode, a música gospel e o sertanejo clássico, não só tem seus gêneros bem estabelecidos, como também remetem a músicas de outras décadas. E no caso da música clássica a outros séculos. O terceiro segmento apresenta um perfil mais estabelecido do que os dois passos anteriores. Visto que cada *playlist* citada está muito bem estabelecida no seu próprio gênero. Na terceira divisão encontramos *playlist* urbanas que em sua maioria surgiram entre periferias da cidade e movimentos sociais. São elas I Love my'90s Hip-Hop, Rapública, Mansio Reggaeton, hits urbanos, Rapcaviar, Rap Workout e Funk Hits, essa temática abrange uma característica importante para indústria fonográfica. Pois o hip-hop e o Funk surgiram dos

movimentos sociais e posteriormente ganharam seu espaço na indústria musical. O quarto segmento se divide em três blocos, no geral os dois blocos a direita abrangem *playlists* com características similares. Pois no geral essas *playlists* estão ligadas ao conteúdo mainstream, ou seja, é conteúdo realmente focado ao público popular. Os chamados hits, são conhecidos como as músicas que estão “na boca do povo”. Esse grupo maior começa com a *playlist* Top Brasil que se encontra ao lado de Funk hits e vai até a *playlist* Dance hits. Um fato curioso é a *playlist* Are & Be, que se refere ao gênero Rhythm and Blues, esse gênero preserva semelhanças mais próximas dos gêneros citados no primeiro e segundo segmento. Porém suas características numéricas fizeram a sua dissimilaridade ficar mais próxima dos outros gêneros, porém ainda sim, essa *playlist* se encontra sozinha, separada das outras. Dentro desse segmento maior, se dividiu um grupo menor com músicas que remetem a gêneros mais alegres e relaxantes, são eles: Reggae Classics, Roots Rising, soft pop hits, Feelin’ Good. E o último segmento restante foram os gêneros derivados do Rock and Roll

7.7 MONA

O algoritmo MONothetic Analysis que é apelidado de Mona é a função de clusterização que mais se diferencia das demais. Isso se dá pois o foco dele é em variáveis binárias onde retorna uma lista hierárquica de *clusters*. Esses *clusters* hierárquicos são formados com um grande *cluster* no início do processamento. O segundo passo é selecionar uma variável e dividir os dados com base no seu valor binário, formando dois grupos. O terceiro passo escolherá uma nova variável entre as que sobraram e repetirá o mesmo processo de divisão, até sobrar um único objeto. Vale mencionar que a variável escolhida é a que apresenta associação máxima às demais variáveis dentro do *clusters* da k-ésima etapa. O algoritmo também tem como requisito que pelo menos uma variável tenha todas as observações completas, caso contrário o algoritmo para. Com isso, a medida de associação que é utilizada é da expressão (26) que é o determinante da tabela de contingência de uma variável f e g genérica. A soma dos elementos do quadro 2 é a quantidade de valores não omissos da variável f , então partindo disso se calcula (26) para cada variável completa de g . Então a associação é determinada para uma variável t . Que é a expressão (27) sendo o

máximo de t para o par de variáveis do determinante da matriz de contingência A das variáveis f e g . Se houver empate para o valor do t , é escolhido o primeiro da lista, ou seja, se dois valores iguais forem o máximo obtido então o primeiro na lista de variáveis será retornado.

$$A_{fg} = |b_{fg}c_{fg} - a_{fg}d_{fg}| \quad (26)$$

Quadro 5 - Exemplo de tabela de contingência

		Variável g	
Variável f		a_{fg}	b_{fg}
		c_{fg}	d_{fg}

$$A_{ft} = \max_t A_{tfg} \quad (27)$$

`mona(x, trace.lev = 0)`

A função recebe assim como as outras funções um argumento x que é uma matriz ou um data frame. Porém, todas as variáveis devem ser binárias. A documentação ainda informa que há um número tolerável de variáveis faltantes, pois na primeira etapa o algoritmo avalia os valores ausentes. Pesquisa-se a maior associação absoluta com outra variável, se a associação é positiva o valor da variável com maior associação é adicionada no valor faltante. Se for negativa é feito 1- (O valor da variável com maior associação). O segundo argumento é

o *trace.lev* que recebe um valor lógico para indicar se o algoritmo deve informar o progresso da saída da função.

Para aplicação dessa função o banco de dados construído na seção de seleção de variáveis não se demonstra muito interessante. Visto que só há duas variáveis dicotômicas, são elas a explícito e a modo. Explícito que se referencia se o conteúdo é classificado como ofensivo e/ou inapropriado para crianças. E o modo que se refere se a escala é maior ou menor. Podemos criar critérios para outras variáveis para torná-las dicotômicas. Assim pode-se criar um banco de dados somente com variáveis binárias e realizar a análise com base nesses critérios.

Com base na análise exploratória a variável popularidade apresenta valores iguais a 0, ou seja, as músicas não se classificam como populares. Então o primeiro critério a ser construído é se a variável popularidade tem sua observação igual a 0, ela receberá 0. Se for maior que 0 ela receberá 1 marcando que houve algum tipo de popularidade. O segundo critério que será criado será com base no tom da música, se o tom for sustenido receberá o valor 0, se for um tom dentro da escala maior de Dó será classificado com o valor 1. O terceiro critério foi em relação ao tipo da postagem da música, ou seja, se ela pertence a um álbum, single ou uma compilação de músicas. O critério será se a música pertence a um álbum receberá o valor 1 se o valor da observação da variável for single ou compilation será atribuído o valor 0. A criação dos critérios está disposta no quadro abaixo:

Quadro 6 - Critérios utilizado para criação de variáveis binárias

Variáveis	0	1
Explícito	False	True
Popularidade	observação = 0	observação > 0
Sustenidos?	observação = 1 ou 3 ou 6 ou 8 ou 10	observação = 0 ou 2 ou 4 ou 5 ou 7 ou 9 ou 11)
tipo	observação = “single” ou “compilation”	observação = “album”
modo	observação = “menor”	observação = “maior”

Abaixo se encontra o algoritmo que realiza a aplicação desses critérios para a utilização da função, será utilizado o pacote ‘*dplyr*’ para as transformações.

Mona - formação do banco e aplicação

Popularidade: se a música consegue algum tipo de popularidade > 0 = 1

```
pop_bin <- data_ws%>%  
  select(popularidade)%>%  
  mutate(popularidade = ifelse(popularidade==0,0,1))
```

Tom é sustenido ou não?

0 = Dó | 1 = Dó# | 2 = Ré | 3 = Ré# | 4 = Mi | 5 = Fá | 6 = Fá# | 7 = G | 8 = G# | 9 = A | 10 = A# | 11 = B

```
sus_bin<-data_ws%>%  
  select(tom...6)%>%  
  mutate(tom...6=ifelse(tom...6==0 |  
                        tom...6==2 | tom...6==4| tom...6==5| tom...6==7 |  
                        tom...6==9 | tom...6==11, 1, 0))
```

Tipo: se a música fizer parte de um álbum o valor é 1 se for single ou compilado recebe 0

```
tipo_bin <- data_ws %>%  
  select(tipo) %>%  
  mutate(tipo=ifelse(tipo=="album",1,0))
```

Ajuste na variável explícito

```
explicito<-data_cat%>%  
  select(explicito)%>%  
  mutate(explicito=ifelse(data_cat$explicito==FALSE,0,1))
```

Criando o data frame com 5 variáveis binárias

```
data_bin <- data.frame(explicito$explicito,  
                      sus_bin$tom...6,
```

```

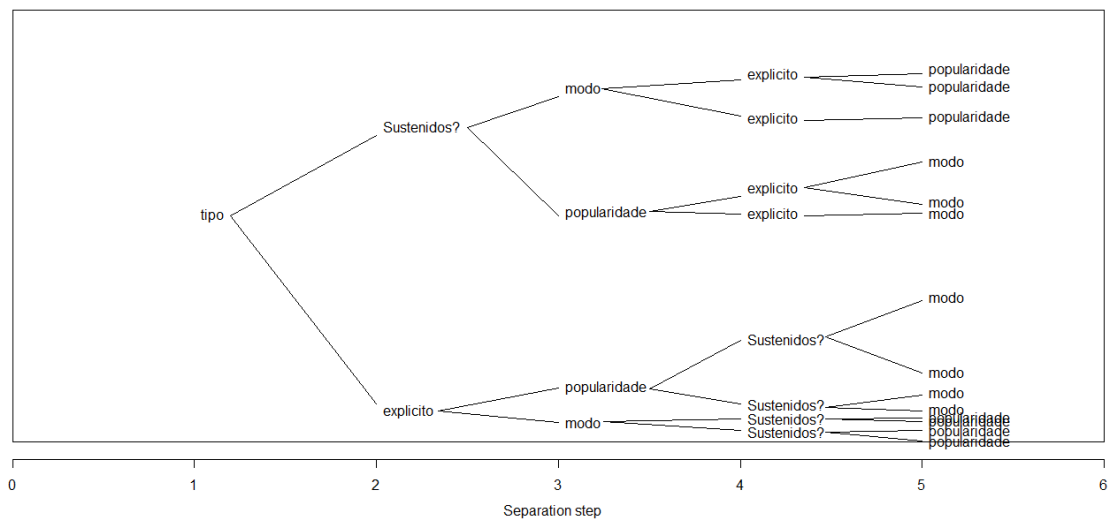
pop_bin$popularidade,
tipo_bin$tipo,data_ws$modo)
names(data_bin)<-c("explicito","Sustenidos?","popularidade","tipo","modo")
# Aplicação da Mona - MONothetic Analysis
mn<-mona(data_bin)
plot(mn)

```

Com o banco de dados construído, agora podemos aplicar o algoritmo Mona, ela retorna um objeto com vários atributos, são eles: data, hasNA, order, variable, step, order.lab e call, porém o mais interessante da função é aplicar o plot com esse objeto.

Figura 25 - Gráfico tipo banner com aplicação da função Mona aos dados binários

Banner of mona(x = data_bin)



Antes de analisar o gráfico gerado, é importante destacar que o gráfico na Figura 26, não é a saída exata do plot com o objeto gerado pela Mona. Acontece que os segmentos de retas não acompanham o plot e foram inseridos para a compreensão das divisões do MONothetic Analysis. As coordenadas das retas podem ser encontradas no código que está no *github* deste trabalho. Então, Analisando o gráfico, temos no eixo x cada etapa que o algoritmo executa, no passo 0 ele uniu todas as variáveis. Foi selecionado o “tipo” como a variável que apresenta maior associação com as outras variáveis. Então o algoritmo separou os álbuns dos singles e

compilações. No segundo passo as variáveis que apresentaram a maior associação dentro dos *clusters* foram “explícito” e “sustenidos?”. No terceiro passo tanto para “explícito” como para o “sustenidos?”. As variáveis que agora apresentam o maior grau de associação absoluta é “popularidade” e “modo” e no quarto passo as variáveis “explícito” e “sustenidos?”. Aparecem de forma alternada, no segmento do passo 2 na variável "explícito", no quarto passo aparece o “sustenido?”. No passo 2 com o segmento de “sustenido?” no passo 4 aparece a variável “explícito”. E no quinto e último passo “modo” e “popularidade” aparecem como as variáveis menos associadas. É importante perceber que as variáveis não se repetem nos seus segmentos de reta. O que torna possível traçar perfis sobre a associação das variáveis. No caso da aplicação deste trabalho, a variável tipo tem a máxima associação com as outras variáveis, o que estabelece o primeiro nível de associações, utilizando a separação de 0's e 1's.

8. CONSIDERAÇÕES FINAIS

A importância de técnicas de agrupamento no momento atual em que se é produzida quantidades massivas de dados retorna a necessidade que tínhamos na antiguidade de agrupar para compreender e resolver problemas. O objetivo específico deste trabalho é servir de guia para a realização de técnicas de agrupamento, compreendendo a importância das suas etapas para uma melhor construção de *clusters*. Assim podendo atingir o objetivo geral que era a compreensão do uso de técnicas de agrupamento dentro do contexto multivariado. Para isso foi adotado o pacote *cluster* que utiliza uma abordagem interessante e clara sobre o conteúdo. Então foi criado um guia e um tutorial sobre a utilização deste pacote utilizando os 7 algoritmos propostos pelo Kaufman e Rousseeuw (1990).

A forma que este trabalho foi construído visa em suas seção de métodos estabelecer os conceitos necessários para qualquer tipo de técnica de clusterização. Posteriormente é focado com o pacote para realização destes conceitos e suas possíveis aplicações. Ilustrando possíveis problemas com os dados originados da *API* do *spotify* que foi construído na seção

banco de dados por meio de Web Scraping. Cada seção das funções DAISY, PAM, CLARA, AGNES, DIANA e MONA foi formada com a seguinte estrutura: introdução com o objetivo da função, seu método e critérios do algoritmo, uma problemática para estabelecer um objetivo para sua aplicação, consequentemente o código com a manipulação dos dados para alcançar o objetivo. Visto que essa etapa é de suma importância e que demanda maior tempo de criação no processo e pouco citada em trabalhos, suas saídas, análises e interpretações dos seus resultados. Com isso é alcançado o objetivo específico do trabalho, explorando técnicas de particionamento, aglomerativas e divisivas.

As aplicações das funções Pam, Clara, Fanny, Agnes e Diana podem servir de auxílio para empresas, como a *spotify* por exemplo. Assim ajudando a compreender o perfil de seus usuários. Como visto na Pam, Clara e Fanny, pode-se identificar grupos tanto em regiões no gráfico, quanto em tabelas com base nas métricas adotadas. Ajudando identificar o gosto do cliente. A Agnes e a Diana também auxiliam nesse processo, podendo identificar gêneros e artistas mais similares. O Caso da Mona com o foco em variáveis binárias ajuda a identificar associações existentes nos dados. Estudos sociais em empresas e nas áreas acadêmicas, utilizam com frequência esse tipo de variável. As aplicações neste trabalho exploraram essas características utilizando dados de diversas fontes de gêneros e artistas. Essas ferramentas focadas em casos específicos, para os usuários podem se tornar ainda mais relevantes para compor o estudo de formação de perfis. Podendo assim melhorar recomendações e distribuições de músicas.

Outros trabalhos interessantes dentro do conteúdo de clusterização é explorar outros tipos de algoritmos e métodos para realizar agrupamentos. Até mesmo criação de guias e tutoriais para as funções base do software *R* para aplicações desse assunto. Indo além da fundamentação e percepção do conteúdo, podem ser feitas aplicações para variados tipos de fontes de dados, explorando as características de cada algoritmo tanto do pacote *cluster*, como de outras funções.

Link para o material complementar (códigos, bancos de dados):

<https://github.com/DuarteSamuel/Monografia>

REFERÊNCIAS

CANBERRA DISTANCE. scientificlib, 2021. Disponível em: <http://www.scientificlib.com/en/Mathematics/LX/CanberraDistance.html>. Acesso em 14 de Set. 2021

COR.TEST: TEST FOR ASSOCIATION/CORRELATION BETWEEN PAIRED SAMPLES, rdr.io R Package Documentation, 2021. Disponível em: <https://rdr.io/r/stats/cor.test.html>. Acesso em 01 de Set. 2021

EVERITT, Brian; HOTHORN, Torsten. **An Introduction to Applied Multivariate Analysis with R**. Springer, 2011º

JOHNSON, Richard A.; WICHERN, Dean W. **Applied Multivariate Statistical Analysis**. Pearson Prentice Hall, 2008.

KAUFMAN, Leonard; ROUSSEEUW, Peter J. **Finding Groups in Data**. An Introduction to Cluster Analysis. John Wiley & Sons, Inc, 1990.

Maechler, M. et al. **cluster: Cluster Analysis Basics and Extensions**. R package version 2.1.2.

MACQUEEN, J. **Some Methods for Classification And Analysis of Multivariate Observations**. Paginas 281-297 do: Fifth Berkeley Symposium on Mathematics, Statistics and Probability. University of California Press. University of California Press, 1967.

MONTGOMERY, Douglas C., Design and Analysis of Experiments, Editora John Wiley & Sons, New York, 2013.

ROUSSEEUW, Peter J. “Silhouettes: a Graphical Aid to the Interpretation and Validation of Cluster Analysis”. Computational and Applied Mathematics. 20: 53-65. Disponível em: <https://www.sciencedirect.com/science/article/pii/0377042787901257?via%3Dihub> . Acesso em 14 de set. 2021

SANTINI, Rose Marie. **As dimensões sociais dos gêneros musicais: porque os sistemas de classificação comercial e não comercial variam**. Transinformação, vol. 25, núm. 2, agosto, 2013, pp. 101-110 Pontifícia Universidade Católica de Campinas Campinas, Brasil.

Disponível em: <https://www.redalyc.org/articulo.oa?id=384334896001>. Acesso em 25 de ago. 2021

STRUYF, Anja; et al. **Clustering in an Object-Oriented Environment**. Department of Mathematics and Computer Science, U.I.A.,Universiteitsplein 1, B-2610 Antwerp, Belgium. Disponível em: <https://www.jstatsoft.org/article/view/v001i04> . Acesso em 18 de ago. de 2021.

APÊNDICE A - CÓDIGO PARA WEB SCRAPING DOS DADOS DO SPOTIFY

A primeira etapa é a instalação da biblioteca, a segunda é a configuração do diretório e das chaves (Client ID e Client Secret), a terceira é a leitura do banco com a lista de *playlists* e seus *URL* para serem extraídas pela função, e o último passo é a aplicação da função `get_playlist_audio_features` para criação do data frame.

Parte 1 - biblioteca

```
Install.packages("spotifyr") # instalação da biblioteca  
Library(spotifyr)           # leitura da biblioteca
```

Parte 2 - Configurações

```
# configurações da chave  
Id<- "Insira a chave Client ID do seu login"  
Secret<- "Insira a chave Cliente Secret do seu login"  
# configuração para acesso  
Sys.setenv(SPOTIFY_CLIENT_ID = id)  
Sys.setenv(SPOTIFY_CLIENT_SECRET = secret)  
access_token <- get_spotify_access_token ()
```

Parte 3 – Leitura do banco de dados com a lista de *playlists*

Dados das playlist que serão extraídas

```
dados_playlist <- read_excel("SpotifyScrap.xlsx")
```

```
dados_playlist <- dados_playlist[, 1:5]
```

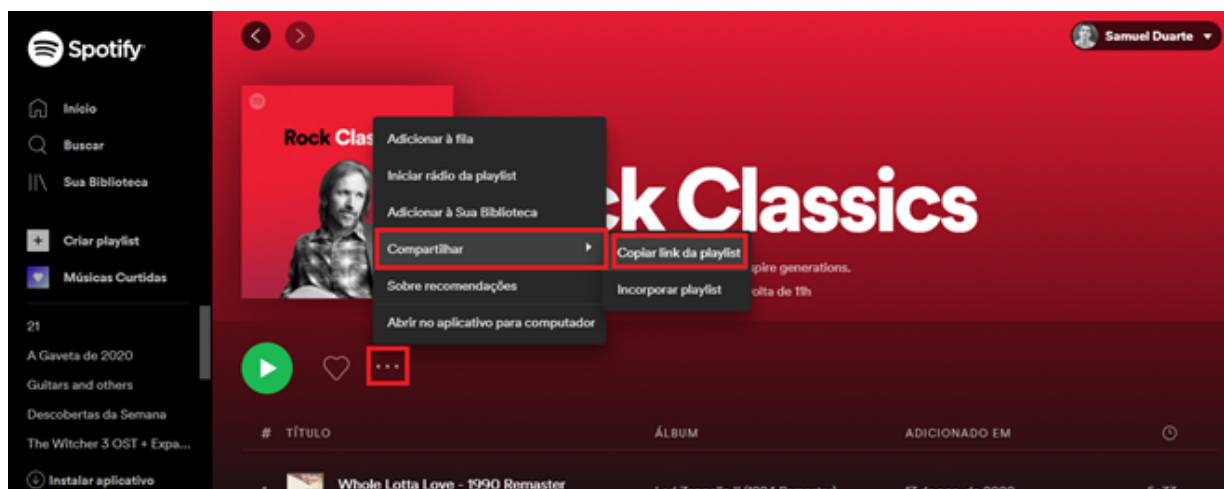
```
head(dados_playlist)
```

A planilha SpotifyScrap.xlsx foi construída manualmente, nela temos cinco campos, um contador, o nome da *playlist*, o código de compartilhamento da *playlist*, o número de músicas presente na *playlist* e por último o número de vezes que a *playlist* foi salva na biblioteca dos usuários. A função `head` retorna a planilha com essas características da Tabela 1.

Tabela apêndice A - Primeiras linhas da planilha para extrair os dados do Spotify

Nº	Nome da playlist	código	faixas	curtidas
1	Roots Rising	37i9dQZF1DWYV7OOaGhoH0?si=d8f5d030bbd34fe0'	107	1824072
2	Old School Metal	37i9dQZF1DX2LTcinqsO68?si=dab657eaa26943d2'	140	2008497
3	Nu Metal Generation	37i9dQZF1DXcfZ6moR6J0G?si=59251247ddf44fab'	70	1093112
4	Rock Classics	37i9dQZF1DWXRqgorJj26U?si=15fa9e799068438f'	145	9159473
5	SoftRock	37i9dQZF1DX6xOPeSOGone?si=588135ecfb47474a'	100	3020302

Figura apêndice A - Página com código de compartilhamento do site do *spotify*



Parte 4 – Extração dos dados com a função do *spotifyr*

```
play_uris <- dados_playlist$código'; data <- get_playlist_audio_features(play_name, play_uris); View(data)
```

APÊNDICE B - SUMARIZAÇÃO DA FUNÇÃO FANNY - COM AS SAÍDAS PARA CADA OBJETO GERADO PELO ALGORITMO

Fuzzy Clustering object of class 'fanny' :

```
m.ship.expon. 2
objective      2.309458e+08
tolerance      1e-15
iterations     74
converged      1
maxit          500
n              12
```

Membership coefficients (in %, rounded):

	[,1]	[,2]	[,3]
A	82	15	3
A#	72	27	1
B	8	90	2
C	25	41	34
C#	13	10	77
D	16	82	2
D#	27	49	23
E	93	6	1
F	6	93	1
F#	3	3	94
G	26	72	2
G#	1	1	98

Fuzzyness coefficients:

```
dunn_coeff normalized
0.6921429    0.5382144
```

Closest hard clustering:

```
A A# B C C# D D# E F F# G G#
1 1   2 2 3 2 2 1 2 3 2 3
```

Silhouette plot information:

	cluster	neighbor	sil_width
A	1	2	0.808795173
E	1	2	0.757203082
A#	1	2	0.568641934
B	2	1	0.605453884
D#	2	1	0.453898062
C	2	1	0.430153749
D	2	1	0.027146742
F	2	1	-0.007622586
G	2	1	-0.331031732
C#	3	1	0.767241987
G#	3	2	0.744553300
F#	3	2	0.644519207

Average silhouette width per cluster:

```
0.7115467    0.1963330    0.7187715
```

Average silhouette width of total data set:

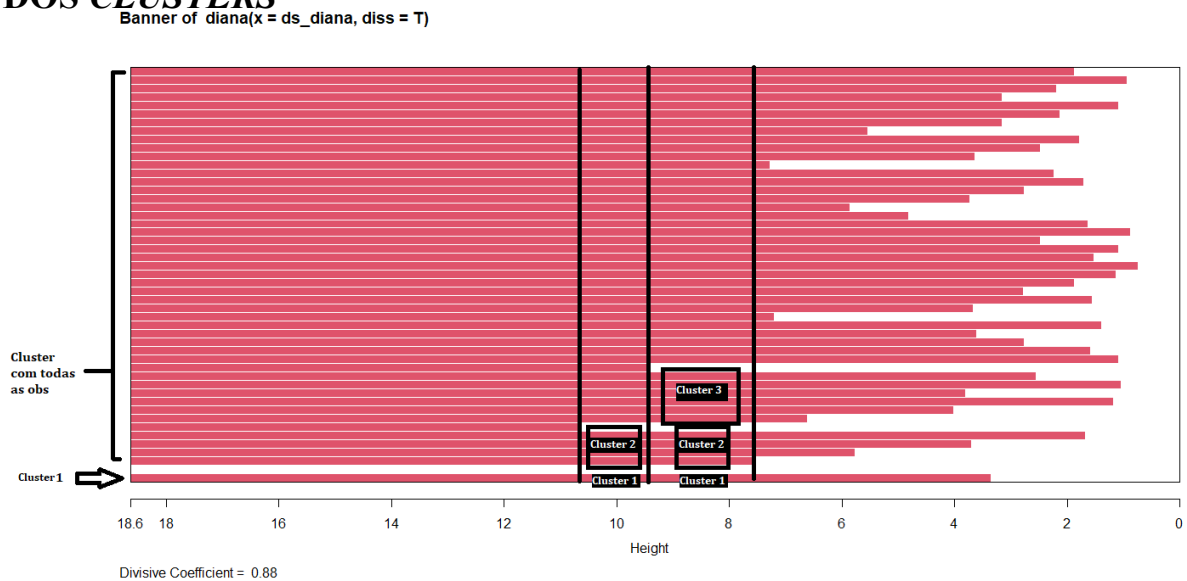
```
0.4557461
```

66 dissimilarities, summarized :

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	
2.799e+06	5.966e+07	1.442e+08	2.427e+08	3.626e+08	1.023e+09	
Metric : SqEuclidean			Number of objects : 12			

Available components: "membership", "coeff", "memb.exp", "clustering", "k.crisp", "objective", "convergence", "diss", "call", "silinfo", "data"

APÊNDICE C - FIGURA EXPLICATIVA COM PLOTAGEM DO BANNER PARA FUNÇÃO DIANA COM O MOMENTO DE DIVISÃO DOS *CLUSTERS*



Foram traçadas linhas verticais representando o momento da divisão e a região que representa o cluster. No gráfico isso segue até o momento que o *cluster* com a menor altura fica isolado. A linha vermelha segue até o momento em que a observação foi dividida dos demais *clusters*. No momento 18.6 da escala Height no canto inferior esquerdo podemos perceber que existe uma linha vermelha, acima dela existe um espaço em branco e depois todas as outras linhas vermelhas. Neste momento existem dois *clusters*, a linha vermelha inferior e todas as linhas que estão separadas do espaço em branco. No momento 10 da escala height podemos observar que umas das linhas vermelhas não continua. até o momento em que outra linha não continuar, ali ocorre a segunda divisão. Agora existem 2 *cluster* formados e 1 *cluster* com todas as observações.